

레몬•Aid

AI 헬스케어 · 건강관리 플랫폼

팀 프로젝트 기획서 · v1.0

2026년 5월

Team Lemon · KDT WebAI

Lemon Aid — AI 헬스케어 / 건강관리 플랫폼

Lemon Aid 팀 · 2026.05 · v1.2

경북대학교 AI/빅데이터 전문가 양성 과정 — (주)레몬헬스케어 협업 프로젝트

한 줄 요약

"사진 한 장으로 끝나는 만성질환 관리 — AI Agent가 내 병원 기록과 생활 데이터를 기억한다."

만성질환자가 음식과 영양제를 사진으로 찍으면, AI Agent가 병원성 데이터·복약 정보·건강 데이터를 함께 해석해 부족 영양소·과다 섭취·주의 성분·식단관리 점수를 개인화해 알려주는 모바일 앱.

핵심 메시지 3개:

- 사진 한 장으로 끝 — 영양제 라벨·음식 촬영 후 5종 분석 자동 출력
- 병원 기록을 기억하는 Agent — 만성질환·복약·검사값을 함께 본다
- AI는 거드는 손길 — 진단·처방 X, 사용자 결정 주도 + 미리보기 후 승인

---SPLIT---

1. 프로젝트 개요

1.1 프로젝트 정체성

항목	내용
팀명	Lemon Aid
프로젝트명	AI 헬스케어 / 건강관리 플랫폼
서비스 가칭	Lemon Aid Agent
포지션	독립형 참조 앱 (건강의신 직접 연동 X, 향후 적용 가능한 방향 제안)
발주처	(주)레몬헬스케어
수행 주체	경북대학교 AI/빅데이터 전문가 양성 과정 — Lemon Aid 팀 (5명)
수행 기간	2026.04.28 ~ (8주 MVP)
결과물	모바일 앱 프로토타입 + 백엔드 API + 분석 알고리즘·3개 Agent 통합 흐름 데모 + LDB 연계 인터페이스 설계서

1.2 발주 배경

(주)레몬헬스케어는 2026년 코스닥 상장과 함께 B2C 건강관리 플랫폼 **건강의신**으로 사업을 확장 중이다. 기존 청구의신은 2025년 12월 종료되었고, 770만+ 누적 사용자 베이스를 건강관리 플랫폼으로 피벗시키고 있다.

회사가 가진 자산은 단순 앱이 아니다. 상급종합병원 80%(37/47개) 점유, 130여 곳 의료기관 네트워크, 표준화 의료데이터 API 180여 종, LDB(Lemon Digital Bridge) 의료데이터 중계 플랫폼. 즉 "AI가 학습할 고품질 의료 데이터를 유통하는 인프라 기업"이다.

이 자산을 살려 만성질환자 중심의 AI 영양·복약 관리 서비스를 어떻게 만들 수 있는가가 본 프로젝트의 출발점이다.

1.3 과제 해석 — 두 개의 출발점, 하나의 도착지

레몬헬스케어가 제시한 과제는 크게 두 개의 축으로 나뉜다.

구분	내용	5종 출력 매핑
과제 #1 — 영양제 분석	복용 중인 영양제 기반으로 개인별 섭취 영양소·권장 기준·부족 영양소 추천·섭취량 권고·목적별 특화 분석	① 부족 영양소 / ② 권장 섭취량 / ⑥ 목적별 분석
과제 #2 — 음식·식단 분석	음식·식단 기반 섭취 영양소 분석 + 활동 정보 결합으로 체중 변화 예측·활동 권고·부족 영양소 추천	① 부족 영양소 / ③ 체중 예측 / ④ 활동 권고

우리는 이를 단순 AI 모델 개발 과제로 보지 않는다. 이 과제는 레몬헬스케어의 **병원 데이터 기반 개인화 헬스케어 목표에 도달하기 위한 Agent형 서비스 실증 과제**로 해석한다.

따라서 음식과 영양제 분석은 각각 독립된 출발점이고, **최종 방향은 두 섭취 데이터를 병원성 데이터와 생활 데이터에 연결한 만성질환자 관리 서비스**다.

1.4 우리가 기획하는 것

이 프로젝트는 단순 음식 분석 앱이나 영양제 정보 앱이 아니다.

핵심은 만성질환자의 **병원성 데이터·복약 정보·음식·영양제 섭취 기록·건강 데이터**를 AI Agent가 함께 이해하고, 사용자에게 맞는 식단관리 판단과 실천 피드백을 제공하는 것이다.

내 병원 기록과 생활 데이터를 기억하는 만성질환자용 AI 영양·복약 관리 Agent

우리는 **건강의신과 직접 연동하지 않는다**. 대신 건강의신에 적용 가능한 독립형 참조 앱을 만들어, 레몬헬스케어가 향후 실제 서비스에 반영할 수 있는 방향을 제안한다.

1.5 페르소나

1차 핵심 — 김건강 (52세, 남성, 만성질환 관리자)

☑52세 남성, 고혈압 진단 2년차, 당뇨 전단계

혈압약 1종 + 영양제 4종 (오메가3·비타민D·종합비타민·밀크씨슬)

178cm / 84kg (BMI 26.5, 비만 1단계)

평일 6,500보, 주말 8,000보

[Job To Be Done]

- 약과 영양제가 충돌하지 않는지 확인

- 만성질환자에게 맞는 영양·운동 권장량

- 합병증 예방 (당뇨로 진행 막기)

- 가족이 안심할 수 있는 일관된 관리

"만성질환을 가진 분이라면 일반 영양제 앱은 충분치 않습니다. 우리는 진료 데이터까지 연결합니다."

2차 확장 — 박직장 (38세, 남성, 예방 단계 직장인)

☑38세 남성, 정기 검진에서 콜레스테롤·공복혈당 경계

175cm / 82kg (BMI 26.8)

평일 5,000보 미만, 시간 부족

영양제 2종 (멀티비타민·오메가3)

[Job To Be Done]

- 최소 시간으로 만성질환을 예방

- 효율적 식단·운동 관리

- 3개월 후 결과를 미리 보고 싶다

"사진 한 장으로 끝나는 건강 관리, 3개월 후 체중·건강을 미리 보세요."

1.6 차별화 포인트 5개

차별 무기	본 프로젝트	경쟁 (필라이즈/CalZen 등)
의료기관 네트워크 연계 가능성	LDB 130여 의료기관 (참조 인터페이스 설계)	X
만성질환 v4 가중 알고리즘	당뇨/고혈압/심혈관/관절/호흡기 가중 (×1.3)	X
검증된 사용자 베이스	770만+ (청구익신 자산)	△
5종 출력 + Agent 통합 해석	사진 1장으로 5종 + 분석 알고리즘·3개 Agent 통합 흐름	△ (단편)
사진 기록 응모권 UX	점수가 아닌 참여 기반, 부담 0	X

출처 메모: 필라이즈 MAU·투자 유치 수치는 시리즈 B 보도자료 기준(2024년 발표). 정확한 인용은 §23 참고 자료 참조.

1.7 일정 개요 (8주 MVP)

상세 주차별 매핑은 §12.3 참조.

단계	기간	핵심
Phase 0 — 기획·환경	W1~W2	병원 데이터 연동 질문 정리 / Figma 와이어프레임 / 환경 셋업
Phase 1 — 코어 구현	W3~W4	음식·영양제 OCR / 분석 알고리즘 / 알고리즘 단위
Phase 2 — Agent 통합	W5~W6	평가·개인화·챗봇 3개 Agent / 알림·캘린더 / 응모권
Phase 3 — QA·시연	W7~W8	통합 QA / 의료자문위 / 배포 / 시연 리허설

---SPLIT---

2. 배경 및 문제 정의

2.1 시장 환경

지표	수치	의미
65세 이상 인구 비율	2025년 20% 돌파	초고령사회 진입, 만성질환자 급증
국내 건기식 시장	2024년 6조 원 (+8% YoY)	영양제 일상화
사용자 평균 영양제 복용 종수	3~5종 동시	중복·과다 위험 상존
필라이즈 MAU	75만, 시리즈 B 120억 유치	추격이 아닌 차별화 필요

2.2 사용자 페인포인트 6가지

만성질환자는 병원 검사·복약·식단·영양제·활동 데이터를 따로 관리한다. 결과로 다음 6가지 문제가 일상화돼 있다.

- 1. **영양제 정보 비대칭** — 성분 표기 형식이 제각각, 영문 라벨이 많아 일반 사용자가 해석 어려움
- 2. **중복·과다 섭취 위험** — 종합비타민과 단일 비타민 동시 복용 시 같은 성분이 합산되어 상한 초과
- 3. **영양 균형 파악 어려움** — 식단과 영양제를 통합해 보여주는 도구 부재
- 4. **인과관계 불투명** — "운동을 늘렸는데 체중이 안 빠진다" 같은 실패 분석이 안 됨
- 5. **만성질환자 맞춤 부재** — 대부분 앱이 일반인 기준만 제시, 당뇨·고혈압자에 맞춘 권장량 X
- 6. **데이터 분산** — 걸음수·식단·영양제·검진 결과가 각기 다른 앱·기록지에 흩어짐

2.3 4가지 가설과 검증 지표

가설	검증 지표 (MVP 기준)
H1: OCR+LLM 파이프라인으로 영양제 등록 부담을 줄일 수 있다	OCR 정확도 ≥85%, 등록 시간 ≤30초 (테스트셋 30장)
H2: v1~v4-KDRIs-7-step 산출식이 임상적으로 타당하다	의료자문위(의사 1+약사 1+영양사 1) 검토 통과
H3: 5종 출력을 한 화면에 통합하면 만성질환자가 이해하기 쉽다	내부 테스터 5명 + 멘토·자문위 3명 정성 피드백 (SUS 정식 측정은 v2)
H4: LDB 의료기관 연계 인터페이스 설계서가 후속 통합의 출발점이 된다	인터페이스 설계서 완성, 회사 검토 OK

베타 30명+ 정량 사용성 테스트는 v2(MVP+) 단계로 이동. 8주 MVP는 알고리즘 정확성·Agent 동작·표현 안전성에 집중.

2.4 갭 분석 — 우리가 직접 정의해야 할 영역

항목	회사 가이드	우리가 추가 정의
활동점수 v1~v4	산출식 명확	그룹 비교용 코호트 DB
권장 영양소 (KDRIs)	19~49세 기준	50대+, 만성질환자 보정
체중 예측	7-step 결정론	사용자 적응형 보정 (Phase 3)
부족 영양소 추천	알고리즘 미정의	식단 → 영양소 매핑 + 결핍 판단
목적별 분석 (눈/간/피로)	정의 없음	식약처 기능성 인정 원료 매트릭스
식단 입력 처리	미정의	텍스트/이미지 → 영양소 변환
영양제 분석	미정의	영양제 CSV DB 우선 + API 보조, OCR/라벨링 결과 매칭
Agent 협력 구조	미정의	분석 알고리즘 + 개인화·평가·챗봇 3개 Agent 통합 흐름

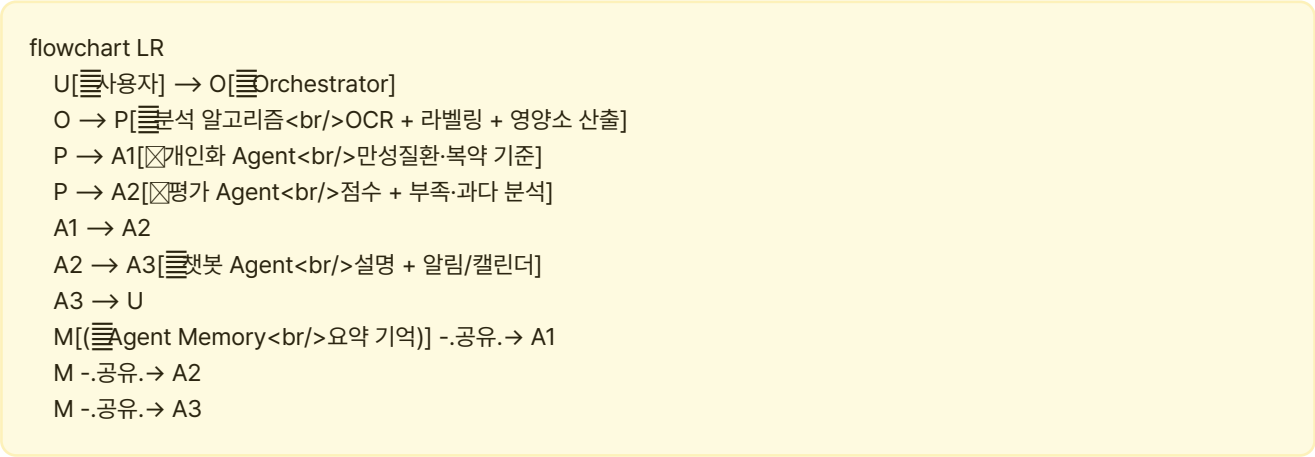
5개 출력 중 4개의 핵심 가치 영역(부족 영양소·목적별 분석·식단·영양제)이 우리가 직접 알고리즘을 설계해야 하는 영역이다. 이것이 사실상 본 프로젝트의 진짜 도전 영역이다.

---SPLIT---

3. 핵심 기능 명세

3.1 분석 알고리즘 + 3개 Agent 통합 구조

단일 LLM 호출이 아니라, 분석 알고리즘이 이미지·자연어·파일 입력을 먼저 구조화하고 개인화·평가·챗봇 3개 Agent가 하나의 통합 흐름으로 만성질환자의 영양·복약 관리를 해석한다.



구성	책임	입력	출력
분석 알고리즘	음식·영양제 사진 인식, OCR, CSV DB/API 매칭, 영양소 산출	사진, 자연어 입력, 파일, 사용자 수정 입력	음식명·섭취량·영양소·영양제 성분/함량 (Pydantic)
개인화 Agent	만성질환·검사값·복약 기준 제공	사용자 프로필, 멘토 확인 전 시연용 데이터	질환별 주의 영양소, 권장 기준, 약물 주의
챗봇 Agent	결과 설명 + 사용자 요청 기반 실행	자연어 질문, 분석 결과, Agent 요약 기억	자연어 답변 + (선택) Tool Call
평가 Agent	식단관리 점수 + 개선 피드백	분석 알고리즘 결과 + 개인화 Agent 기준	점수, 부족 영양소, 과다 위험, 좋은 선택, 개선 필요

분석 알고리즘은 `backend/src/algorithms/`, `ocr/`, `supplements/` 가 담당하고, 3개 Agent의 코드 책임 매핑은 §14 파일 구조의 `backend/src/agents/` 참조.

3.2 Agent 간 데이터 전달 포맷

3개 Agent는 다음 Pydantic 모델로 통신한다. 오케스트레이터가 분석 알고리즘 결과를 받아 직렬/병렬 호출을 결정한다.

```

class AgentInput(BaseModel):
    user_id: int
    request_id: str      # 한 사용자 요청 단위로 동일
    payload: dict        # 분석 결과, 파일/텍스트/숫자, Agent별 입력
    context: AgentMemorySnap # 최근 검사값/만성질환/복약 요약

class AgentOutput(BaseModel):
    request_id: str
    agent_name: Literal['personalization', 'chat', 'evaluation']
    result: BaseModel    # Agent별 전용 결과 모델
    used_tools: list[str] # Tool Use 호출 이름 목록
    latency_ms: int
    cost_usd: float      # 비용 추적

```

오케스트레이터는 각 Agent 호출을 `agent_runs` 테이블에 로깅하여 비용·지연·실패율을 추적한다. Agent 요약 기억 (`agent_memory`) 갱신은 평가 Agent가 끝난 직후 `backend/src/agents/memory.py` 모듈이 담당한다.

3.3 Tool 정의 (LLM Tool Use 함수 목록)

`backend/src/llm/tools.py` 에 정의되며 챗봇 Agent와 분석 알고리즘 흐름이 호출한다.

Tool 이름	호출 주체	인자	효과
extract_supplement_facts	분석 알고리즘	{ ocr_text }	OCR 텍스트를 SupplementParseResult Pydantic으로 강제 파싱
add_reminder	챗봇	{ type, name, time, recurrence, weekdays? }	DB INSERT 후 flutter_local_notifications에 등록
add_calendar_event	챗봇	{ date, time, title, hospital?, note? }	DB INSERT 후 add_2_calendar로 시스템 캘린더 반영
log_supplement_intake	챗봇	{ supplement_id, taken_at }	영양제 섭취 기록 + 응모권 카운트
explain_deficiency	챗봇	{ nutrient, ratio }	부족 영양소 설명을 자연어로 (의료법 표현 검수 후)

모든 Tool 호출은 `backend/src/utils/regex_filter.py` 의 `check_forbidden_terms()` 를 거쳐 사용자에게 전달된다.

3.4 인증·온보딩 흐름

flowchart TD
S[Splash] → L{로그인?}
L -- No → SU[회원가입/로그인
이메일·구글·카카오·간편 로그인]
SU → EV[이메일 또는 소셜 인증]
EV → CN[개인정보 동의
필수/선택 분리]
CN → P1[기본정보
나이·성별·키·몸무게]
P1 → P2[만성질환 다중선택
당뇨/고혈압/심혈관/관절/호흡기/없음]
P2 → P3[복약 입력
없음 가능]
P3 → P4[목적 선택
눈/간/피로/체중]
P4 → P5[건강 데이터 권한
HealthKit/Health Connect]
P5 → P6[알림 권한
거부 시 배너 안내]
P6 → H[홈 진입]
L -- Yes → H

이메일 인증 메일 발송은 백엔드의 `backend/src/services/email.py` 모듈에서 SMTP(개발) 또는 AWS SES/ NCP Cloud Outbound Mailer(운영)로 처리한다. 구글·카카오 등 간편 로그인은 소셜 인증 결과를 받아 동일한 프로필 등록 흐름으로 진입한다.

3.5 주요 화면

화면	핵심 인터랙션	담당 Agent
온보딩 / 프로필	만성질환·복약·기본정보 입력 (§3.4 흐름)	개인화
카메라 (음식·영양제)	사진 촬영, AI 인식, 사용자 수정	분석 알고리즘
5종 출력 대시보드	요약 + 상하 스크롤 세부내용, 부족 영양소 / 권장 섭취량 / 체중 예측 / 활동 권고 / 목적별 분석	분석 알고리즘+개인화+평가
챗봇 화면	자연어 대화, 설명, 알림/캘린더 등록	챗봇
식단관리 점수	끼니별·하루별 점수 + 개선 피드백	평가
응모권 현황	사진 기록 참여 일수 + 누적 응모권	(Agent X, 규칙 기반)
건강 데이터	걸음수·체중·심박 시계열 차트	(Agent X, 시각화)
설정 (로그아웃·탈퇴·동의 관리)	동의 철회·계정 삭제·데이터 내보내기	—

3.6 MVP 핵심 흐름

```
sequenceDiagram
    participant U as 사용자
    participant App as Flutter
    participant API as FastAPI
    participant AL as 분석 알고리즘
    participant AG as Agents
    U->>App: 사진 촬영 또는 자연어 입력
    App->>API: multipart upload 또는 text input
    API->>AL: OCR+라벨링+영양소 산출
    AL->>API: 영양소 JSON
    API->>AG: 개인화 Agent (DB 조회)
    AG->>API: 질환별 주의 기준
    API->>AG: 평가 Agent
    AG->>API: 점수 + 부족·과다·주의
    API->>App: 5종 출력 JSON
    App->>U: 미리보기
    U->>App: 수정·승인
    U->>App: "8시 알림 등록"
    App->>API: chat/message
    API->>AG: 챗봇 Agent (Tool Use)
    AG->>API: add_reminder 호출
    API->>App: 미리보기
    U->>App: 승인
    App->>API: reminders POST
    API->>App: 등록 완료
```

3.7 영양제 분석 보완 흐름

1. 영양제 제품명·라벨·성분표 촬영
2. 분석 알고리즘: 제품명·성분명·카테고리·함량·1회 섭취량·권장 횟수 OCR 분석
3. 사용자가 실제 섭취량·빈도·복용 시간 수정
4. 분석 알고리즘: 영양제 성분을 표준명으로 정리하고 영양소 단위로 환산한 뒤 음식 영양소와 합산
5. 개인화 Agent: 권장 기준·질환·검사값·복약 정보 참고
6. 평가 Agent: 부족·과다·중복·주의 성분 설명
7. 챗봇 Agent: 눈건강·간기능·피로회복 등 목적별 관리 방향 제시 (특정 제품 추천 X)

3.8 사진 기록 응모권 UX

점수 기반이 아니라 **참여 기반**. 식단관리 점수가 낮은 날에도 사진을 찍으면 응모권은 받는다.

조건	응모권
하루 음식·영양제 사진 기록 완료	1개
1주일 연속 기록 완료	+1개
1개월 전체 기록 완료	+3개

예시 보상: 1등 1명 2박 3일 숙박권 / 2등 3명 국내 호텔 숙박권 / 3등 10명 건강식품 5만원권.

캐싱:

- SHA-256 해시는 결과 캐싱 키로 사용하되, 같은 사진 재분석 자체는 차단하지 않음
- 하루 최대 1개 + 계정당 월 8개 상한
- 응모권 발급은 서버 측 먹등키(`user_id + date`)로 강제

식단관리 점수는 응모권 지급 조건이 아니라, 사용자가 자신의 식단을 이해하고 개선하기 위한 피드백 지표다.

3.9 Agent 요약 기억

Agent는 전체 원본 데이터를 모두 저장하지 않고, 개인화에 필요한 핵심 정보만 요약 기억한다. 갱신 시점은 평가 Agent 종료 직후 + 명시적 프로필 변경 직후 (`backend/src/agents/memory.py` 의 `update_memory()` 호출).

요약 기억 대상: 최근 검사값 요약 / 만성질환 태그 / 복약 정보 요약 / 음식·영양제 기록 요약 / 건강 데이터 요약 / 식단관리 점수 / 영양제 섭취 달성 이력 / 응모권 누적 이력 / 최근 주의사항.

3.10 에러·예외 화면 정책

에러 유형	사용자 화면	액션
네트워크 오프라인	"인터넷 연결을 확인해 주세요" 토스트	[수동 입력] [재시도]
OCR 실패 (저정확도)	미리보기에 "라벨이 흐릿해요" 배너	[다시 촬영] [수동 입력]
LLM 타임아웃 (12초)	"분석에 시간이 걸리고 있어요" 스피너	[다시 불러오기] [기본 분석으로 진행]
의료법 표현 차단 (재시도 3회 실패)	"분석 결과를 만들 수 없어요. 수동으로 입력해 주세요"	[수동 입력]
OCR/이미지 인식 후보 실패	분석을 다시 불러오거나 수동 입력으로 전환	[다시 불러오기] [수동 입력]
알림 권한 거부	"챗봇이 등록된 알림이 동작하지 않아요" 배너	[설정으로 이동]

3.11 오프라인 정책

- 사진 촬영은 항상 가능. 오프라인이면 `lsar pending_uploads` 큐에 저장
- 네트워크 복귀 감지 시 자동 업로드(연결 유형 무관)
- 충돌(서버에 같은 시각 분석 존재): 서버 우선, 로컬은 "병합 필요" 라벨

- 챗봇-5종 출력은 온라인 전용 — 오프라인 시 "분석은 인터넷 연결이 필요해요" 안내

3.12 멀티디바이스 동기화

서버가 단일 진실. 클라이언트는 last-write-wins + ETag로 충돌을 감지. 응모권 발급은 서버 측 먹등키(`user_id + date`)로 두 기기 동시 사용 시에도 중복 발급 방지.

---SPLIT---

4. 기술 스택 — 프론트엔드

4.1 프레임워크 · 언어

분류	기술	선택 이유	앱에서의 역할
프레임워크	Flutter 3.24+	단일 코드로 iOS+Android 동시 배포로 학생 팀 인력 부담을 줄임. Hot Reload로 UI 빠른 반복	모든 화면(카메라·대시보드·챗봇·프로필) 렌더링
언어	Dart 3.x	강타입, 비동기 친화, Python 사고와 유사	비즈니스 로직, 상태 관리
상태 관리	Riverpod 2.x	Provider 대비 컴파일 타임 안전성, 테스트 용이	사용자 프로필·분석 결과·Agent 응답 캐시 전역 관리
API 통신	Dio + Retrofit	인터셉터·재시도·타임아웃 일관 처리. Retrofit으로 백엔드 API 클래스 자동 생성	백엔드 FastAPI 호출 (영양제 분석·식단·챗봇 메시지)
라우팅	go_router	선언적 라우팅, 딥링크 지원	7화면 전환
이미지 캡처	image_picker + camera	카메라 직접 호출 + 갤러리 다중 선택, 품질·해상도 제어, 최대 15장 재촬영 흐름	영양제 라벨·음식 사진 캡처
헬스 데이터	health 패키지 11.x	iOS HealthKit 우선 연동. Android Health Connect는 검토 예정이며 Play Console 데이터 타입 신청·승인 리스크를 확인	걸음수·심박·체중·활동에너지 온디바이스 수집
차트	fl_chart	시계열·바·도넛 모두 지원, 한글 라벨 안정적	체중 예측·활동 점수·영양소 충족률 시각화
로컬 저장	Isar / Hive	빠른 NoSQL, 오프라인 우선	분석 결과 캐시, 오프라인 큐 (<code>pending_uploads</code>), 응모권 카운트
푸시 알림	flutter_local_notifications	로컬 알림으로 서버 푸시 없이 동작. iOS는 권한 요청, Android 13+는 POST_NOTIFICATIONS 런타임 권한	챗봇이 등록한 복약·식단 알림 트리거
캘린더	add_2_calendar	시스템 캘린더에 진료 일정 등록	챗봇이 등록한 진료 일정을 사용자 캘린더에 반영
모델	freezed + json_serializable	불변 데이터 클래스, 자동 직렬화	API 응답 파싱, 상태 데이터

4.2 UX 원칙

만성질환자(주로 50대+)도 어렵지 않게 쓸 수 있는 UX가 코어. 화려함보다 명확함.

원칙	적용
큰 글씨와 충분한 여백	본문 16px+, 핵심 17~20px, 터치 영역 최소 48dp
3-탭 이내 핵심 동작	카메라 → 결과 → 저장 흐름이 3탭 안에 끝
숫자 대신 색·그래프	"비타민D 35%" 대신 "충분 / 약간 부족 / 부족" 색 라벨 병기
결과 설명은 쉬운 말	"권장량의 0.35배" 대신 "권장량보다 적게 드셨어요"
AI 결과 미리보기 후 승인	분석 결과·알림 등록은 항상 사용자가 확인 후 저장
MVP는 라이트 단독	디자인 토큰은 다크 대비도 준비, 다크 모드 정식 출시는 v2
법적 안전 표현	"진단" "처방" 표현 X, "주의가 필요할 수 있습니다" "전문가 상담을 권장합니다"

4.3 화면 동작 흐름 (예: 영양제 등록)

[홈] → FAB(+) 탭
↓
[카메라] 영양제 라벨 촬영
↓ image_picker
[로딩] "분석 중..." 스피너 (2.5~6초)
↓ Dio로 백엔드 호출
[미리보기] 인식된 제품명·성분·함량 표시
↓ 사용자 수정 가능
[승인] "이대로 저장" 버튼
↓ Riverpod에 반영 + Isar 캐시
[결과] 5종 출력 대시보드로 이동
↓ 챗봇 진입 가능
[챗봇] "이 영양제 계속 먹어도 돼?" 질문 → 설명 응답

---SPLIT---

5. 기술 스택 — 백엔드

5.1 프레임워크 · 언어

분류	기술	선택 이유	앱에서의 역할
언어	Python 3.11+	팀 익숙도, 데이터·AI 라이브러리 풍부, Type Hint 성숙	모든 비즈니스 로직과 알고리즘
프레임워크	FastAPI 0.110+	Pydantic 기반 자동 OpenAPI/Swagger, async/await가 OCR/LLM I/O 바운드에 강점, Uvicorn workers로 병렬	REST API 서빙, 인증, AI 호출 오케스트레이션
검증	Pydantic v2	LLM 응답·외부 API 응답 강제 검증	Claude 응답 스키마 강제, 요청 바디 검증
인증	JWT + python-jose	Stateless 인증, 모바일 친화	이메일·구글·카카오·간편 로그인 토큰, idToken 검증
비동기 작업	asyncio + asyncio.gather	영양제 사진 여러 장 병렬 OCR, LLM 호출 동시성	run_full_analysis에서 영양제 N장 동시 처리
백그라운드 작업	FastAPI BackgroundTasks	Agent 메모리 갱신·이메일 발송·통계 집계	응답 후 비차단 처리
메일 발송	aiosmtp (개발) / boto3 SES 또는 NCP API (운영)	환경변수 EMAIL_PROVIDER로 분기	회원가입 이메일 인증
테스트	pytest + pytest-cov + httpx + pytest-asyncio	50+ 단위 테스트, 알고리즘 결과를 가이드 PPT 예시값과 일치 검증	v1~v4·7-step·KDRIs 수치 정확성 보장
품질	Black + Ruff + mypy + pre-commit	협업 일관성, 타입 안정성	모든 PR 자동 검증

5.2 핵심 모듈 책임

```

backend/src/
├─ algorithms/      # v1~v4 활동점수, 7-step 체중예측, BMR/TDEE, 결핍 판단, 목적별
├─ ocr/             # OCR/이미지 인식 후보 어댑터
├─ llm/             # Claude/OpenAI 클라이언트, 시스템 프롬프트, Tool 정의, 스키마
├─ nutrition/       # KDRIs 룩업 + 영양소 합산
├─ prediction/      # 체중 예측 보정·시계열 분석
├─ activity/        # 걸음수·심박 → 활동점수 산출
├─ supplements/     # 영양제 파서 + 식약처 DB 매치
├─ agents/          # 개인화·평가·챗봇 3개 Agent + 오케스트레이터 + agent_runs 로깅 +
memory.py
├─ services/        # email.py(메일 발송), storage.py(이미지 저장)
├─ api/             # FastAPI 라우터
├─ models/          # SQLAlchemy ORM
├─ schemas/         # Pydantic 요청·응답
├─ db/              # 세션·init.sql
├─ cache/           # Redis 래퍼
└─ utils/           # 해시·정규식 검수·로거

```


5.3 API 엔드포인트 (전체 목록)

엔드포인트	메서드	설명
/api/v1/auth/signup	POST	회원가입 (이메일+비밀번호), 인증 메일 발송
/api/v1/auth/verify-email	GET	이메일 인증 토큰 검증
/api/v1/auth/login	POST	이메일 로그인 → access + refresh JWT
/api/v1/auth/social	POST	구글·카카오 등 간편 로그인 → access + refresh JWT
/api/v1/auth/refresh	POST	토큰 갱신
/api/v1/auth/logout	POST	로그아웃 (refresh 무효화)
/api/v1/auth/account	DELETE	탈퇴/전체 삭제 요청 (3개월 복구 가능 기간 후 완전 삭제)
/api/v1/profile	GET / PUT	사용자 프로필·만성질환·복약
/api/v1/profile/consent	POST	동의 항목 별도 저장·철회
/api/v1/supplements/analyze	POST (multipart)	영양제 사진 → 분석 결과
/api/v1/supplements	GET / DELETE	등록된 영양제 목록·삭제
/api/v1/meals/analyze	POST (multipart)	음식 사진 → 분석 결과
/api/v1/meals/manual	POST	식단 텍스트 입력
/api/v1/meals	GET / DELETE	식단 기록 CRUD
/api/v1/analysis/full	POST	5종 출력 통합 분석
/api/v1/chat/message	POST	챗봇 메시지 → 응답 + (선택) 액션
/api/v1/reminders	GET / POST / DELETE	복약·식단 알림 CRUD
/api/v1/calendar/events	GET / POST / DELETE	진료 일정 CRUD
/api/v1/health/sync	POST	HealthKit 우선 동기화, Health Connect는 검토
/api/v1/score/daily	GET	식단관리 점수 조회
/api/v1/raffle/status	GET	응모권 누적 현황
/api/v1/agent/memory	GET	Agent 요약 기억 조회 (디버깅·투명성용)
/api/v1/data/export	GET	사용자 데이터 전체 내보내기 (개인정보 권리)

5.4 알고리즘 모듈 동작 흐름

flowchart TD; R[FastAPI 라우터] --> J[JWT 검증]; J --> O[Agent 오케스트레이터]; O --> AN[분석 알고리즘
OCR + 라벨링]; O --> PE[개인화 Agent
DB 조회]; AN --> AL[알고리즘 모듈
v4 / 7-step / KDRIs]; PE --> AL; AL --> EV[평가 Agent]; EV --> MEM[Agent Memory 갱신
BackgroundTasks]; EV --> SE[Pydantic 직렬화]; SE --> JS[JSON 응답];

---SPLIT---

6. 기술 스택 — 데이터베이스

6.1 데이터베이스 구성

분류	기술	선정 이유	앱에서의 역할
메인 DB	PostgreSQL 16	JSON/JSONB(영양제 동적 스키마), GIN 인덱스 (식품 폴텍스트 검색), 컬럼 단위 AES-256 암호화로 의료 데이터 보안	사용자·영양제·음식·진단·식단 기록 영속
시계열 확장	TimescaleDB 2.x (<code>timescale/timescaledb:latest-pg16</code> 이미지)	Hypertable로 걸음수·체중·심박 효율 처리, 자동 다운샘플링·압축. 첫 부팅 시 <code>init.sql</code> 에서 <code>CREATE EXTENSION IF NOT EXISTS timescaledb;</code> 자동 실행	HealthKit 우선, Health Connect 검토 시계열 데이터
캐시 / 큐	Redis 7	OCR 결과 캐싱(SHA-256 해시 키), KDRIs 룩업 캐싱, 이미지/API rate limiting, 세션. 동일 사진 재분석은 차단하지 않음	영양제 분석 캐시 (TTL 30일), 챗봇 컨텍스트 캐시
파일 스토리지	S3 호환 (MVP는 NCP Object Storage 또는 로컬 디스크)	사용자 동의 후 영양제·음식 원본 사진 저장. 처방·검사표 이미지는 멘토 확인 후 검토	<code>photo_url</code> 필드의 실제
마이그레이션	Alembic (async)	SQLAlchemy 기반, 버전 관리	스키마 변경 추적
컨테이너	Docker Compose	로컬 개발 환경 통일	<code>timescale + redis</code> 한 번에 부팅

6.2 핵심 테이블

```

users                # 사용자(uid, email, email_verified_at, name, created_at,
deleted_at)
profiles             # 프로필(user_id, age, gender, height, weight,
#                   chronic_diseases[] AES-256, medications[] AES-256,
goals[])
consents             # 동의 이력(user_id, type, accepted_at, revoked_at)
supplements          # 영양제 CSV 기반 마스터(id, product_name_ko/en, manufacturer,
category)
supplement_ingredients # 영양제 성분(supplement_id, name_ko/en, standard_name, amount,
unit)
user_supplements     # 사용자 복용(user_id, supplement_id, dose, frequency)
foods                # 식품 마스터(식약처 + 농진청 데이터 임포트)
meals                # 끼니 기록(user_id, date, meal_type, foods[], photo_url)
analysis_results     # 분석 결과(user_id, date, meal_type, deficiencies[] AES-256,
#                   excesses[], warnings[] AES-256, goal_analysis)
# ※ 끼니별 단위. 점수는 daily_scores에 단일 저장.
daily_scores         # 하루 점수(user_id, date, score, breakdown, agent_comment)
reminders            # 복약·식단 알림(user_id, type, time, recurrence, weekdays[],
active)
calendar_events      # 진료 일정(user_id, date, time, title, hospital, note)
raffle_tickets       # 응모권 누적(user_id, earned_at, count, reason,
idempotency_key)
agent_memory         # Agent 요약 기억(user_id, summary_json, updated_at)
agent_runs           # Agent 호출 로그(request_id, agent_name, latency_ms, cost_usd,
status)
audit_logs           # PHI 접근·수정 감사 로그(user_id, actor, action, target,
created_at)
email_verifications  # 이메일 인증 토큰(user_id, token, expires_at, verified_at)

# 시계열(TimescaleDB Hypertable)
step_counts          # 걸음수(user_id, ts, count)
weight_logs          # 체중(user_id, ts, kg)
heart_rate_samples   # 심박(user_id, ts, bpm)

```

6.3 보안 · 컴플라이언스

항목	적용
민감정보 컬럼 암호화	<code>chronic_diseases</code> , <code>medications</code> , <code>analysis_results.deficiencies</code> , <code>analysis_results.warnings</code> AES-256
전송 구간	TLS 1.3 강제
Row Level Security	PostgreSQL RLS로 본인 <code>user_id</code> 데이터만 접근
감사 로그	의료 정보(PHI) 조회·수정 모두 <code>audit_logs</code> 테이블 기록
백업 암호화	<code>pg_dump</code> 결과 GPG 암호화 후 보관
삭제 / 동의 철회	사용자 탈퇴·삭제 요청 시 전체 삭제 처리, 3개월 복구 가능 기간 후 완전 삭제

보안 셋업 책임자: D(백엔드) — JWT, RLS, AES-256 컬럼 암호화 모두 D 담당. 컴플라이언스 검토는 E(데이터·도메인) 협업.

6.4 캐싱 전략 (3단계)

- 1. **Redis L1:** OCR 결과(영양제 사진 SHA-256 → 분석 JSON), TTL 30일. 동일 사진 재분석 차단은 하지 않음
- 2. **Redis L2:** KDRIs 룩업·식약처 기능성 인정 원료, TTL 영구 (수동 무효화)
- 3. **PostgreSQL:** 사용자별 분석 결과 영속, Agent 요약 기억

캐싱으로 LLM·OCR 호출 비용을 50% 이상 절감.

---SPLIT---

7. AI · 인공지능 스택

본 프로젝트의 AI 보조 기능을 구현하기 위한 모든 기술을 한 곳에 정리한 섹션. 사용자의 주도권을 해치지 않는 선에서, 음식·영양제 인식은 분석 알고리즘이 담당하고 개인화 판단·챗봇 설명·식단 평가는 3개 Agent가 통합 흐름으로 분담한다.

7.1 모델 (LLM Provider)

모델	용도	선정 이유
Anthropic Claude (모델 ID는 <code>CLAUDE_MODEL_ID</code> 환경변수, 기본값은 최신 Sonnet)	분석 결과 구조화 보조 + 3개 Agent 추론 엔진 — 개인화 판단, 챗봇 응답, 평가 코멘트	비정형 텍스트 → JSON 구조화에 강점, 한국어 우수, 200K 컨텍스트, Tool Use가 Pydantic 스키마와 직접 통합, 의료 표현 안전성
OpenAI GPT (백업)	Claude 장애 시 폴백	OpenAI SDK 동일 패턴, Adapter 패턴으로 교체
text-embedding-3-small (선택)	식품·영양제 유사도 검색	사용자 입력 "비타민C" → 마스터 DB 매칭

7.2 이미지 인식·OCR 후보

도구	용도	선정 이유
Google Cloud Vision API	영양제 라벨·음식 메뉴판 OCR 후보	DOCUMENT_TEXT_DETECTION 기반 OCR 후보. 실제 적용 전 비용과 정확도를 비교
YOLOv8	음식 사진 객체 인식 후보	음식 사진 인식 가능성 검토 및 논문 조사 대상
Naver CLOVA OCR (백업)	한국어 라벨 폴백	한국어 SOTA, Cloud Vision 장애 또는 저정확도 케이스 대응

7.3 분석 알고리즘과 3개 Agent 역할 · 동작

분석은 Agent가 아니라 알고리즘·OCR·CSV DB/API 매칭 파이프라인으로 처리한다. 개인화·챗봇·평가 3개 Agent는 Claude 단일 모델을 쓰되, 전용 시스템 프롬프트와 Tool Use 스키마가 다르다. 코드 위치는 `backend/src/agents/<agent_name>_agent.py` .

7.3.1 분석 알고리즘

항목	내용
책임	음식·영양제 사진 → 영양소 산출
입력	사진(multipart), 자연어 입력, 파일, 사용자 수정 입력
출력	MealAnalysisResult / SupplementParseResult Pydantic 스키마
흐름	이미지 검증·전처리 → OCR/인식 후보 적용 → 영양제 CSV DB 우선 매칭 + API 보조 → 제품명·성분명·카테고리 표준화 → 영양소 환산
Tool 호출	extract_supplement_facts (구조화 보조)
기법	Structured Output (JSON Schema), 규칙 기반 단위 변환
사용자 효과 과	영양제 라벨 한 장으로 30~60초 안에 성분 자동 정리

7.3.2 개인화 Agent (`personalization_agent.py`)

항목	내용
책임	사용자 만성질환·검사값·복약 정보를 해석 기준으로 변환
입력	사용자 프로필 (DB), 시연용 검사값, 복약 정보
출력	PersonalizationContext (질환별 주의 영양소, 권장 기준, 약물 주의)
흐름	DB 조회 → Claude로 "이 사용자가 주의할 영양소·약물 상호작용" 요약 → 캐시
기법	Few-shot Prompting + Context Summarization
사용자 효과	"당뇨가 있으니 탄수화물 주의" 같은 맞춤 권고가 자동 적용

7.3.3 챗봇 Agent (`chat_agent.py`)

항목	내용
책임	분석 결과를 쉬운 말로 설명 + 사용자 요청 기반 알림·캘린더 등록
입력	자연어 질문, 분석 결과, Agent 요약 기억
출력	자연어 답변 + (선택) Tool Call (add_reminder, add_calendar_event, log_supplement_intake, explain_deficiency)
흐름	의도 분류 → 설명 vs 실행 분기 → Tool Use로 알림/캘린더 등록
Tool 호출	add_reminder, add_calendar_event, log_supplement_intake, explain_deficiency
기법	Function Calling, Streaming, Chain-of-Thought
사용자 효과	"매일 아침 8시에 혈압약 알림 맞춰줘" 한 줄로 알림 등록

7.3.4 평가 Agent (`evaluation_agent.py`)

항목	내용
책임	끼니별/하루별 식단관리 점수 + 부족·과다·주의 분석
입력	분석 알고리즘 결과 + 개인화 Agent 기준
출력	EvaluationResult (점수, 부족 영양소, 과다 위험, 좋은 선택, 개선 필요)
흐름	영양소 합산 → KDRI/UL 비교 → "주의 가능성", "권장량 대비", "전문가 상담 권장" 형식으로 자연어 코멘트 생성 → BackgroundTask로 agent_memory 갱신
기법	Structured Output + Few-shot
사용자 효과	"점심은 단백질이 부족했어요. 저녁에 닭가슴살 어떠세요?"

7.4 적용 LLM 기법

기법	적용 Agent	앱에서의 역할	사용자 효과
Tool Use / Function Calling	분석 알고리즘, 챗봇	LLM 출력을 우리 함수(extract_supplement_facts, add_reminder)로 직접 매핑. 텍스트가 아닌 앱 액션 실행	"비타민D 매일 9시 알람"으로 즉시 등록
Structured Output (JSON Schema)	분석 알고리즘, 평가	응답을 정해진 Pydantic 스키마로 강제, 누락·오타 자동 거부	영양제 분석이 빈칸·오타 없이 항상 동일 형식
Few-shot Prompting	개인화, 평가	시스템 프롬프트에 만성질환자 사례 2~3개를 박아 튜닝·구체성 학습	"단백질 부족"이 아니라 "당뇨가 있으니 닭가슴살·두부 추천"처럼 구체적
Chain-of-Thought	챗봇	"복약 안내 → 부작용 주의 → 전문가 상담 권유"처럼 단계 사고 지시	복잡한 영양제 질문도 빠짐없이 답변
Context Summarization	개인화	Agent 요약 기억 사용, 매번 전체 데이터 안 보냄	비용·지연 ↓, 같은 사용자에게 일관된 답변
Streaming	챗봇, 평가	응답을 토큰 단위로 흘러 보여줌	5~8초 대기를 타이핑 애니메이션으로 체감 단축
Forbidden Term Filter	모든 Agent	LLM 응답을 정규식으로 후처리, 진단·처방·치료 등장 시 차단·재생성	의료법 위반 표현 0건

7.5 보조 알고리즘 (Non-LLM)

알고리즘	적용 기능	앱에서의 역할	사용자 효과
KDRIs 록업 테이블	권장 섭취량	나이·성별·BMI 키로 30종 영양소 RDI를 즉시 반환 (LLM 불필요)	화면 진입 즉시 권장량 표시
결핍 판단 비율 분류	부족 영양소 추천	실제/RDI 비율로 5단계(결핍/낮음/적정/과다/위험) 분류	"비타민D 35%" 같은 객관적 수치
v1~v4 활동 점수	운동 권고	결정론 공식, 만성질환 가중 ×1.3까지	"당신의 7,000보는 일반인의 9,000보 가치"
7-step 체중 예측	체중 변화	Mifflin-St Jeor BMR + 활동계수 + 7,700 kcal/kg	1주/1개월/3개월 후 체중 미리보기
충돌 감지 규칙	영양제 중복·과다 경고	같은 성분이 합산되어 UL(상한) 초과 시 경고	종합비타민과 단일 비타민 동시 복용 시 경고
응모권 누적 규칙	사진 기록 참여	일별 기록 카운트, 1/7/30일 조건 충족 시 자동 발급. SHA-256은 캐싱에 사용하고 동일 사진 재분석은 차단하지 않음	점수 부담 없이 참여만으로 응모권

7.6 비용 · 성능 가드레일

항목	목표
모델	Claude 단일 (장애 시 OpenAI 폴백)
비용 산식	분석 1건 = OCR \$0.0015 + LLM 입력 ~3K 토큰 + 출력 ~1K 토큰 ≈ \$0.024
PoC 비용 가정	베타 5명 × 분석 5회/주 × 4주 = 100건 → 약 \$2.4 + 챗봇 일평균 3회 ≈ 월 \$10
정식 비용 가정	MAU 1만 × 평균 5분석 + 3챗봇 → 약 \$1,200/월 (캐시 적중률 50% 가정)
캐싱 3단계	Redis(OCR 30일) / Redis(KDRIs 영구) / PostgreSQL(사용자별 분석 결과). 동일 사진 재분석 차단은 하지 않음
레이트 리밋	사용자당 분당 5회, 일당 50회
타임아웃	OCR 8초, LLM 12초, 합산 응답 6초 이내 (캐시 미스 기준). 제한 초과 시 다시 불러오기 또는 수동 입력 폴백
max_tokens	분석 1024 / 평가 800 / 챗봇 600 / 개인화 400
장애 대응	LLM 실패 시 친절한 에러 + 수동 입력 폴백 (\$3.10)

7.7 프롬프트 거버넌스

- 모든 시스템 프롬프트는 `backend/src/llm/prompts.py` 한 곳
- 각 프롬프트마다 버전 태그 (`analysis_v1`, `chat_v2` 등)
- 한국어 출력만 허용 (영어 섞임 방지)
- 의료법 금지 표현 사전 차단 시스템 프롬프트 + 사후 정규식 필터 이중화
- AI는 결과를 직접 저장하지 않는다, 항상 사용자 미리보기 후 승인 필수

---SPLIT---

8. 핵심 알고리즘

회사 가이드 8개(BMI·v1~v4·BMR·TDEE·7-step) + 우리가 직접 설계한 4개(영양제 OCR·식단 변환·결핍 판단·목적별 분석).

8.1 알고리즘 지도

flowchart TB
P[사용자 프로필] → BMI[BMI 분류]
P → BMR[BMR Mifflin-St Jeor]
BMR → TDEE[TDEE = BMR × 활동계수]
INTAKE[섭취칼로리·일수] → WP[7-step 체중 예측]
TDEE → WP

STEP[걸음수·심박·만성질환] → V[v1 → v2 → v3 → v4]

SUP[영양제 사진] → OCR[OCR + Claude + 식약처 매칭]
MEAL[식단 사진/텍스트] → NM[음식 영양소 합산]
OCR → SUM[총 섭취 영양소]
NM → SUM
SUM → DEF[결핍 판단 5단계]
P → DEF

GOAL[사용자 목적] → GM[목적별 매트릭스]

8.2 BMI 분류 (한국·아시아 기준)

구분	BMI
저체중	< 18.5
정상	18.5 ~ 22.9
과체중	23.0 ~ 24.9
비만 1단계	25.0 ~ 29.9
비만 2단계	≥ 30.0

8.3 활동점수 v1 ~ v4

$v1 = \min(\text{실제걸음} / \text{권장걸음}, 1.2) \times 83.33$
 권장걸음 = $8000 \times \text{성별계수}(\text{여 } 0.95 / \text{남 } 1.0)$
 $\times \text{나이계수}(<40: 1.0 / 40\sim59: 0.9 / 60+: 0.8)$
 $\times \text{BMI계수}(\text{저 } 0.9 / \text{정 } 1.0 / \text{과 } 1.05 / \text{비1: } 1.1 / \text{비2: } 1.15)$

$v2 = v1 \times (0.7 + 0.3 \times \text{심박계수})$
 심박계수 = $\min(\text{목표심박 유지분} / 30, 1.0)$
 웨어러블 미착용 시 0.7
 목표심박 = $(220 - \text{나이}) \times 0.5 \sim 0.7$

$v3 = \min(100, v2 + \text{백분위 보너스})$
 상위 10% / 20% / 30% → +10 / +5 / +3
 표본 < 30명이면 0

$v4 = \min(100, v3 \times \text{만성질환가중})$
 가중 = $1.0 + \Sigma(\text{당뇨/고혈압} +0.10 / \text{심혈관/관절} +0.15 / \text{호흡기} +0.10)$
 최대 1.3

검증 예시 (50대 여성, 비만 1단계, 7,000보, 당뇨+고혈압): $v1 = 77.5 \rightarrow v2 \approx 69.7 \rightarrow v3 = 72.7 \rightarrow v4 = 87.2$

8.4 7-step 체중 예측

1. BMR (Mifflin-St Jeor)
 남: $10W + 6.25H - 5A + 5$
 여: $10W + 6.25H - 5A - 161$
2. TDEE = BMR × 활동계수 (걸음수 기반)
 < 5,000: 1.2 (좌식)
 ~ 7,500: 1.375 (가벼운 활동)
 ~ 10,000: 1.55 (중간)
 ~ 12,500: 1.725 (활발)
 ≥ 12,500: 1.9 (매우 활발)
3. 일일 수지 = 섭취칼로리 - TDEE
4. N일 누적 = $\Sigma(\text{일일 수지})$
5. 이론 변화 = 누적 / 7,700 kcal/kg
6. 현실 보정: 감량 × 0.85 / 증량 × 0.95
7. 예측 체중 = 시작 체중 + 보정 변화

검증 예시 (50세 여성 160cm/68kg, 6,500보, 1,500kcal, 30일): BMR 1,269 → TDEE 1,745 → 일일 -245 → 누적 -7,350 → 이론 -0.955 → 보정 -0.81 → **67.19 kg**

8.5 영양제 OCR → CSV DB/API 매칭 파이프라인

- 이미지 검증·전처리 (용량 제한, 회전 보정, 흐림 감지, 해상도 제한)
 - 카메라: 최대 15장까지 다시 촬영 가능
 - 갤러리: 다중 선택 가능
 - 실패 시 은행 앱 신분증 촬영처럼 다시 촬영 또는 수동 입력
- SHA-256 해시 → Redis 캐시 조회 (TTL 30일)
 - └─ 히트 → 즉시 반환
 - └─ 미스 ↓
- OCR/이미지 인식 후보 적용 → raw 텍스트 또는 음식 후보
- Claude Tool Use (extract_supplement_facts, Pydantic 스키마 강제)
 - {product_name, serving_size, ingredients[{name_ko, name_en, amount, unit, daily_value_pct}]}
- 영양제 CSV DB 우선 매칭 + 외부 API 보조 연결
 - 제품명·성분명·카테고리 분류
 - 표준명이 있으면 표준화 ("비타민 C" = "Vitamin C" = "ascorbic acid")
 - mg, µg, IU 등 단위 변환
 - DB/API 미매칭 시 사용자 수정 가능
- PostgreSQL INSERT + Redis SET (TTL 30일)
- JSON 응답

8.6 결핍 판단 로직

실제 섭취량 = $\Sigma(\text{음식 영양소}) + \Sigma(\text{영양제 영양소})$
 $RDI = KDRIs \cdot \text{록업}(\text{나이} \cdot \text{성별} \cdot \text{BMI} \cdot \text{만성질환 보정})$
 비율 = 실제 / RDI

비율 분류:

< 0.35	→ DEFICIENT	(결핍)
0.35~0.7	→ LOW	(낮음)
0.7~1.3	→ ADEQUATE	(적정)
1.3~UL	→ EXCESSIVE	(과다)
> UL	→ RISKY	(위험)

결핍 영양소는 비율 낮은 순으로 priority 정렬

8.7 목적별 분석 매트릭스 (식약처 기능성 인정 원료만)

목적	핵심 영양소	권장량	근거
눈건강	루테인+지아잔틴	10~20 mg/일	AREDS2 황반변성 임상
눈건강	오메가-3 (DHA)	1~2 g	망막·안구건조
간기능	밀크씨슬 (실리마린)	≥130 mg	식약처 "간 건강" 인정
간기능	NAC	600~1,800 mg	글루타티온 전구체
피로회복	비타민 B1/B2/B12	KDRIs 기준	에너지 대사
피로회복	CoQ10	90~200 mg	미토콘드리아 ATP
피로회복	마그네슘	KDRIs 기준	ATP 합성·근육

경고 분기: 흡연자 + 눈건강 → 루테인 폐암 위험 경고 자동 표시

8.8 검증 4단계

레벨	방법
L1	단위 테스트 50+개, 가이드 PPT 예시값과 ±0.1 이내 일치
L2	알고리즘 모듈 통합 테스트
L3	의료자문위 검토 (의사 1+약사 1+영양사 1)
L4	내부 테스터 5명 + 멘토·자문위 3명 정성 피드백 (정량 SUS는 v2)

---SPLIT---

9. AI 호출 흐름

9.1 영양제 사진에서 결과까지

```
sequenceDiagram
    autonumber
    participant U as 사용자
    participant App as Flutter
    participant API as FastAPI
    participant R as Redis
    participant V as OCR/인식 후보
    participant C as Claude
    participant DB as PostgreSQL

    U->>App: 사진 촬영
    App->>API: multipart upload
    API->>API: SHA-256 해시
    API->>R: 캐시 조회
    alt 캐시 히트
        R->>API: 분석 JSON
    else 캐시 미스
        API->>V: OCR/이미지 인식
        V->>API: raw 텍스트
        API->>C: Tool Use extract_supplement_facts
        C->>API: Pydantic JSON
        API->>DB: CSV DB 우선 매칭 + API 보조
        DB->>API: 정규화 결과
        API->>DB: INSERT 분석 결과
        API->>R: SET (TTL 30일)
    end
    API->>App: JSON 응답
    App->>U: 미리보기
    U->>App: 수정·승인
    App->>API: 5중 출력 요청
```

합계: 캐시 미스 2.5~6초 / 캐시 히트 1초 미만

9.2 챗봇으로 알림 등록 (Tool Use)

```
sequenceDiagram
    autonumber
    participant U as 사용자
    participant CA as 챗봇 Agent
    participant API as FastAPI
    participant DB as PostgreSQL

    U->>CA: "매일 아침 8시 혈압약 알림"
    CA->>CA: Claude + tools=[add_reminder]
    CA->>CA: tool_call: add_reminder(...)
    CA->>CA: Pydantic 검증 + 의료법 검수
    CA->>API: POST /reminders
    API->>DB: INSERT
    DB->>API: 등록 OK
    API->>U: "8시 혈압약 알림 등록했어요"
```

핵심 원칙:

- API 키는 백엔드 환경변수에만 (모바일 앱 빌드물에 노출 X)
- LLM 응답은 항상 Pydantic 검증 + 의료법 표현 검수 후 사용자 미리보기
- AI는 결과를 바로 저장하지 않는다, 항상 사용자 승인 후 실행

9.3 시스템 프롬프트 예시 (분석 구조화 보조)

당신은 영양제 라벨 분석 전문가입니다.
입력으로 OCR 텍스트를 받아 `extract_supplement_facts()` 함수를
정확한 JSON 인자로 호출하세요.

규칙:

1. 한국어 성분명을 우선으로 하되 영어명도 함께 기록.
2. 함량 단위 통일 (mg, mcg, IU).
3. "1일 권장량 대비 %"가 명시된 경우 `daily_value_pct` 채움.
4. 인식 불가능한 항목은 빈 값. 추측 금지.
5. "복용해도 됩니다" 같은 단정적 표현 사용 금지.
6. 의료법 위반 표현(진단·처방·치료) 사용 금지.

---SPLIT---

10. 데이터 활용 & 외부 API

10.1 공공 데이터 (모두 무료/저비용)

데이터	출처	용도
KDRIs 2020	한국영양학회 / 보건복지부	30종 영양소 권장 섭취량, BMI별 칼로리 조정
식품영양성분 Open API	식약처	음식별 영양소 매칭
영양제 CSV DB	팀 구축	제품명·성분명·카테고리 라벨링 우선 기준
건강기능식품 원료 DB	식약처	영양제 성분 매칭 보조, 기능성 표현
국가표준식품성분표	농촌진흥청 국립농업과학원	식품 영양소 보강
AI Hub 한국 음식 이미지	NIA	음식 인식 모델 학습 (Phase 3, 비상업 학습)

10.2 향후 병원 데이터 연동 검토

병원 데이터 연동 기반은 멘토에게 전체 질문지로 남긴다. 이 항목은 MVP 확정 구현으로 쓰지 않는다.

검토 질문:

- 향후 검사값·진단명 태그·복약 정보·진료 요약 중 어떤 데이터를 가정할지
- 실제 병원 데이터 없이 시연용 대체 데이터를 돌지
- 민감정보 동의와 저장 범위를 어디까지 돌지
- 처방·검사표 이미지를 허락받은 뒤 추후 포함할지

10.3 건강 데이터 연동

플랫폼	데이터	우선순위	권한 셋업
Apple HealthKit (iOS)	걸음수, 체중, 심박	우선 연동	Info.plist에 NSHealthShareUsageDescription 사유 명시, App Store 심사 정당화
Google Health Connect (Android)	걸음수, 체중, 심박	검토 예정	Play Console에서 Health Connect 데이터 타입 신청 (승인 5~10영업일)
혈압	(양 OS 모두)	가능하면 자동, 안 되면 수동 입력	—

10.4 외부 API

API	용도	비고
Anthropic Claude API	분석 결과 구조화 보조 + 3개 Agent 추론	주력 LLM, 모델 ID는 환경변수
Google Cloud Vision API	영양제·음식 OCR 후보	비용·정확도 비교 후 적용
YOLOv8	음식 사진 인식 후보	논문 조사와 기술 검토 대상
Naver CLOVA OCR (백업)	한국어 라벨 풀백	Adapter 패턴
OpenAI API (백업)	LLM 풀백	Claude 장애 대비
AWS SES 또는 NCP Outbound Mailer	이메일 인증 발송	운영 환경

---SPLIT---

11. 데이터 모델

모든 사용자 데이터는 user_id로 격리되어 PostgreSQL에 저장. 시계열은 TimescaleDB Hypertable, 캐시는 Redis.

11.1 사용자 (User · Profile)

```

type User = {
  id: int (PK);
  email: string (unique);
  password_hash: string?;
  social_provider: 'google' | 'kakao' | 'apple' | 'email'?; # iOS 출시 시 Apple 필수
  social_subject: string?;
  display_name: string;
  email_verified_at: timestamp?;
  created_at: timestamp;
  last_login_at: timestamp;
  deleted_at: timestamp?; # 3개월 복구 가능 기간 후 완전 삭제
}

type Profile = {
  id: int (PK);
  user_id: int (FK);
  age: int;
  gender: 'M' | 'F';
  height_cm: float;
  weight_kg: float;
  chronic_diseases: string[]; # AES-256
  medications: string[]; # AES-256
  goals: string[];
}

type Consent = {
  id: int (PK);
  user_id: int (FK);
  type: 'privacy' | 'ai_usage' | 'health_data' | 'image_storage' | 'notifications';
  accepted_at: timestamp;
  revoked_at: timestamp?;
}

type EmailVerification = {
  id: int (PK);
  user_id: int (FK);
  token: string;
  expires_at: timestamp;
  verified_at: timestamp?;
}

```

11.2 영양제 · 식단

```

type Supplement = {
  id: int (PK);
  product_name_ko: string;
  product_name_en: string;
  manufacturer: string;
  serving_size: string;
}

type SupplementIngredient = {
  id: int (PK);
  supplement_id: int (FK);
  name_ko: string;
  name_en: string;
  amount: float;
  unit: 'mg' | 'mcg' | 'IU';
  daily_value_pct: float?;
}

type UserSupplement = {
  id: int (PK);
  user_id: int (FK);
  supplement_id: int (FK);
  dose: float;
  frequency_per_day: int;
  taken_times: string[];
  started_at: date;
}

type Food = {
  id: int (PK);
  name_ko: string;
  name_en: string;
  nutrients_per_100g: jsonb;
  source: '식약처' | '농진청';
}

type SupplementCsvImport = {
  id: int (PK);
  source_file: string;
  imported_at: timestamp;
  row_count: int;
}

type Meal = {
  id: int (PK);
  user_id: int (FK);
  date: date;
  meal_type: 'breakfast' | 'lunch' | 'dinner' | 'snack';
  foods: jsonb;
  photo_url: string?;
}

```

11.3 분석 · 점수 · 알림

```

type AnalysisResult = {
  id: int (PK);
  user_id: int (FK);
  date: date;
  meal_type: 'breakfast' | 'lunch' | 'dinner' | 'snack';
  deficiencies: jsonb;      # AES-256
  excesses: jsonb;
  warnings: jsonb;         # AES-256
  goal_analysis: jsonb;
}

type DailyScore = {
  id: int (PK);
  user_id: int (FK);
  date: date;
  score: int;              # 0~100
  breakdown: jsonb;
  agent_comment: string;
}

type Reminder = {
  id: int (PK);
  user_id: int (FK);
  type: 'medication' | 'meal' | 'supplement';
  name: string;
  time: string;
  recurrence: 'daily' | 'weekly' | 'once';
  weekdays: int[]?;
  active: boolean;
}

type CalendarEvent = {
  id: int (PK);
  user_id: int (FK);
  date: date;
  time: string;
  title: string;
  hospital: string?;
  note: string?;
  added_to_system_calendar: boolean;
}

type RaffleTicket = {
  id: int (PK);
  user_id: int (FK);
  earned_at: timestamp;
  count: int;
  reason: 'daily' | 'weekly_streak' | 'monthly_complete';
  idempotency_key: string;
}

type AgentMemory = {

```

```

id: int (PK);
user_id: int (FK);
summary: jsonb;
updated_at: timestamp;
}

type AgentRun = {
id: int (PK);
request_id: string;
user_id: int (FK);
agent_name: 'personalization' | 'chat' | 'evaluation';
status: 'success' | 'fail' | 'fallback';
latency_ms: int;
cost_usd: float;
created_at: timestamp;
}

type AuditLog = {
id: int (PK);
user_id: int (FK);
actor: 'user' | 'admin' | 'system';
action: string;
target: string;
created_at: timestamp;
}

```

11.4 시계열 (TimescaleDB Hypertable)

```

type StepCount = {
user_id: int (FK);
ts: timestamp;
count: int;
}

type WeightLog = {
user_id: int (FK);
ts: timestamp;
kg: float;
}

type HeartRateSample = {
user_id: int (FK);
ts: timestamp;
bpm: int;
}

```

11.5 보안 · 권한

PostgreSQL Row Level Security (RLS) 적용

```
CREATE POLICY user_isolation ON profiles
FOR ALL
USING (user_id = current_setting('app.user_id')::int);
```

모든 테이블 동일 패턴 적용

컬럼 단위 AES-256: chronic_diseases, medications, deficiencies, warnings

TLS 1.3 강제, 백업 GPG 암호화

---SPLIT---

12. 작업 파이프라인

12.1 단계별 흐름 (한눈에)

flowchart TB

```
S1[1. 아이디어 수립] → S2[2. 차별화 전략]
S2 → S3[3. 디자인 UX → UI]
S3 → S4[4. 프론트엔드 Flutter]
S3 → S5[5. 백엔드 / DB FastAPI]
S3 → S6[6. AI 통합 분석 알고리즘 + 3개 Agent]
S4 → S7[7. 통합 · QA 의료자문위]
S5 → S7
S6 → S7
S7 → S8[8. 배포 TestFlight]
S8 → S9[9. 시연 · 발표 · 회고]
```

12.2 단계별 상세

1단계 — 아이디어 수립

- 목적: 만성질환자의 어떤 불편을 해결할지 한 문장으로
- 활동: 시중 영양제 앱 비교, 페르소나 인터뷰, 갭 분석
- 산출물: 한 줄 요약, 페르소나 2명, 차별화 5가지
- 체크포인트: 30초 안에 컨셉 설명 가능?

2단계 — 차별화 전략 수립

- 목적: 분석 알고리즘 + 3개 Agent 통합 흐름, 응모권 참여 UX, 의료기관 연계 가능성 질문 정리
- 활동: 분석 알고리즘과 Agent 책임 분담, 응모권 규칙 합의, LDB 연계 질문 정리
- 산출물: §3 핵심 기능 명세, §7 AI 스택, §9 호출 흐름

- 체크포인트: 경쟁사가 못 따라하는 자산 명확?

3단계 — 디자인 (UX → UI)

- 목적: 만성질환자(50대+)도 쉽게 쓸 화면
- 활동: 7화면 와이어프레임, 디자인 토큰, Figma 시안
- 산출물: Figma 파일, tokens.dart 초안
- 체크포인트: 50대 사용자 테스트 통과?

4단계 — 프론트엔드 개발 (Flutter)

- 목적: Figma 시안을 실행작 앱으로
- 활동: 환경 셋업, 라우팅, 토큰 적용, 7화면 구현, health 패키지 연동, 알림·캘린더
- 산출물: mobile/ 디렉터리 전체
- 체크포인트: iOS+Android 양쪽 동작?

5단계 — 백엔드 / DB 구축

- 목적: 알고리즘 + API + 데이터 영속
- 활동: Docker Compose, Alembic, 알고리즘 단위, 50+ 테스트, 인증·RLS·암호화
- 산출물: backend/ 디렉터리
- 체크포인트: 가이드 예시 케이스 통과?

6단계 — AI 통합 (4·5와 병렬)

- 목적: §7 AI 스택을 분석 알고리즘 + 3개 Agent로 실행작
- 활동: Claude SDK 래퍼, 3개 Agent 프롬프트, Tool 정의, OCR/이미지 인식 후보 어댑터, 의료법 검수, 미리보기
- 산출물: backend/src/llm/, agents/
- 체크포인트: 같은 입력 → 캐시 적중? 금지 표현 0건?

7단계 — 통합 · QA · 의료자문위 검토

- 목적: 4·5·6 결과 합쳐 끝까지 동작하는 한 흐름
- 활동: 시연 시나리오 3개, 디바이스 테스트, 접근성, 성능, 에러 핸들링, 카피, 의료자문위, TestFlight 첫 빌드
- 산출물: QA 체크리스트, 의료자문위 의견서
- 체크포인트: 시나리오 3개 끊김 없이 시연 가능?

8단계 — 배포

- 목적: 발표장에서 실기기 시연
- 활동: 백엔드 클라우드, TestFlight/Play Internal, 환경 변수, 도메인, 데모 데이터 시드
- 산출물: 라이브 API URL, TestFlight 링크
- 체크포인트: 발표장 와이파이로 3초 내 응답?

9단계 — 시연 · 발표 · 회고

- 목적: 8주 결과를 명확한 메시지로 전달
- 활동: 시연 대본, 데모 데이터, 슬라이드, 백업 플랜, 회고
- 산출물: 슬라이드, 시연 영상, 회고 문서
- 체크포인트: 청중에게 30초 안에 차별화 설명 가능?

12.3 일정 매핑 (8주)

주차	단계	주요 작업
W1	1-2	아이디어/차별화 합의, 본 가이드 작성, Android Health Connect 검토
W2	3	Figma + 디자인 토큰 + 7화면 와이어프레임, 환경 셋업 시작
W3	4-5	Flutter 환경 + 백엔드 환경 + 알고리즘 단위 + Alembic init
W4	4-5-6	카메라·OCR·분석 알고리즘 통합
W5	4-5-6	5종 출력 대시보드 + 평가 Agent + KDRIs 결핍 판단
W6	4-6	챗봇 Agent + Tool Use 알림/캘린더 + 응모권
W7	7	통합 QA + 의료자문위 검토 + TestFlight 첫 빌드 업로드
W8	8-9	배포 + 데모 데이터 + 시연 리허설 + 슬라이드

12.4 단계 간 의존성과 위험 신호

- 3단계 1주 이상 지연 → 4단계는 와이어프레임 PNG로 시작, UI 다듬기는 W6로
- 5단계 막힘 → 시계열 데이터는 v2로 미루고 PostgreSQL 단독 운영
- 6단계 막힘 → 챗봇 Agent 실행 기능은 수동 폼으로 풀백
- 7단계 TestFlight 외부 그룹 심사 미통과 → 내부 테스터(최대 100명) 그룹으로 풀백
- 8단계 막힘 → 발표 PC에서 로컬 백엔드 + 시뮬레이터 직접 시연

원칙: 3개 Agent 중 일부가 단순화되어도 코어(영양제 분석 알고리즘 + 5종 출력)는 살아남는다. 그래서 발표는 무조건 성립한다.

---SPLIT---

13. 사용 툴 정리

본 프로젝트에 실제로 사용하는 도구·서비스·라이브러리를 한 곳에 모아 정리.

13.1 디자인 도구

도구	용도	비고
Figma	UI 시안, 디자인 토큰 정의, 팀 리뷰	Pro 팀 워크스페이스
Figma MCP 서버	Claude(Cowork)가 Figma 파일 직접 분석	디자인-기획 자동 연결
Pretendard	한글 본문/제목 단일 폰트	jsDelivr CDN, 무료 오픈 폰트
Material Symbols	아이콘 세트	Flutter 기본

13.2 AI · LLM 도구

도구	용도	비고
Anthropic Claude (개발 보조)	본 가이드 작성, 3개 Agent 프롬프트 설계, 코드 보조	Cowork (데스크톱) 환경
Anthropic Claude (런타임)	분석 구조화 보조 + 3개 Agent 추론	모델 ID는 환경변수
OpenAI API	LLM 풀백	Claude 장애 시 Adapter 교체
Google Cloud Vision API	영양제·음식 OCR 후보	비용·정확도 비교
YOLOv8	음식 사진 인식 후보	논문 조사와 기술 검토
Naver CLOVA OCR	OCR 풀백	한국어 SOTA

13.3 프론트엔드 (Flutter)

라이브러리	용도	비고
Flutter 3.24+	UI 프레임워크	iOS+Android 단일 코드
Riverpod	상태 관리	컴파일 타임 안전
go_router	라우팅	7화면 전환
Dio + Retrofit	HTTP 클라이언트	인터셉터·재시도
image_picker / camera	카메라·갤러리	영양제·음식 촬영
health	HealthKit 우선, Health Connect 검토	iOS 기준 연동 예정
Isar / Hive	로컬 NoSQL	캐시·오프라인 큐
fl_chart	차트	체중 예측·점수 시각화
flutter_local_notifications	로컬 알림	복약·식단 리마인더
add_2_calendar	시스템 캘린더 등록	진료 일정
freezed	불변 데이터 클래스	모델 정의
json_serializable	JSON 직렬화	API 응답 파싱

13.4 백엔드 · 데이터

도구	용도	비고
Python 3.11+	백엔드 언어	Type Hint 성숙
FastAPI 0.110+	웹 프레임워크	async, Swagger 자동
Uvicorn	ASGI 서버	workers로 병렬
Pydantic v2	스키마 검증	LLM 응답·요청 강제
SQLAlchemy 2.x	ORM	async 지원
Alembic	DB 마이그레이션	버전 관리
PostgreSQL 16	메인 DB	JSONB·GIN·암호화
TimescaleDB 2.x	시계열 확장	Hypertable
Redis 7	캐시·rate limit	OCR/LLM 캐싱
anthropic SDK	Claude API	공식 Python SDK
google-cloud-vision SDK	OCR 후보	공식 Python SDK
aiosmtp / boto3 SES	이메일 발송	EMAIL_PROVIDER 분기
httpx	HTTP 클라이언트	async 지원
python-jose	JWT	Stateless 인증
passlib + bcrypt	비밀번호 해싱	표준
pytest + pytest-cov + httpx + pytest-asyncio	테스트	50+ 단위 테스트

13.5 개발 환경

도구	용도	비고
VS Code / Cursor / Windsurf	코드 에디터	개인 선택
Claude Code / Codex / Cline	바이브 코딩 툴	개인 선택 (PROJECT_GUIDE.md 공통 참조)
Git + GitHub	버전 관리	Lemon-Aid-KDT/Lemon-sin
Black + Ruff + mypy	Python 품질	pre-commit hooks
dart format + flutter analyze	Dart 품질	CI 자동
Docker + Docker Compose	로컬 환경	timescale + redis
Postman / Insomnia	API 테스트	Swagger와 함께

13.6 협업 · 운영

도구	용도	비고
Notion / 구글 닥스	회의록 · 아이디어	팀 커뮤니케이션
카카오톡 / Slack	실시간 소통 + 작업 잠금 공지	충돌 예방
GitHub Actions	CI (backend / mobile / docs)	자동 검증
GitHub Projects	칸반 보드	이슈 트래킹
GitHub Secrets	API 키·환경변수 보관	CI에서 사용
NCP / AWS / GCP	백엔드 배포	학생 크레딧
TestFlight (iOS)	베타 배포	무료, 외부 심사 24~48h
Google Play Internal Testing	베타 배포	무료
OBS Studio (선택)	시연 영상 녹화	백업
QR Generator	발표 시 라이브 URL → QR	청중 즉시 접속

13.7 향후 도입 검토

도구	시점	용도
Sentry	베타 이후	프로덕션 에러 모니터링
Playwright + Patrol	안정화	E2E 테스트 자동화
i18n	글로벌 확장	영문·일문
FHIR KR Core	LDB 통합	표준 의료 데이터 교환

---SPLIT---

14. 파일 구조

전체 디렉터리 트리. 5명이 git clone 직후 빌드·실행 가능한 구조.

```

Lemon_Aid/
├── README.md
├── PROJECT_GUIDE.md      # 본 문서 (마크다운 단일 진실)
├── guide.html            # 같은 내용의 브라우저 뷰어
├──
├── mobile/              # Flutter 앱
│   ├── lib/
│   │   ├── main.dart
│   │   ├── app.dart      # 라우팅 · 부트스트랩
│   │   ├──
│   │   ├── screens/
│   │   │   ├── splash_screen.dart
│   │   │   ├── auth/
│   │   │   │   ├── login_screen.dart
│   │   │   │   ├── signup_screen.dart
│   │   │   │   ├── verify_email_screen.dart
│   │   │   │   └── consent_screen.dart
│   │   │   ├── onboarding_screen.dart
│   │   │   ├── camera_screen.dart
│   │   │   ├── dashboard_screen.dart
│   │   │   ├── chat_screen.dart
│   │   │   ├── score_screen.dart
│   │   │   ├── raffle_screen.dart
│   │   │   ├── health_screen.dart
│   │   │   └── settings_screen.dart
│   │   ├──
│   │   ├── widgets/
│   │   │   ├── supplement_preview.dart
│   │   │   ├── ai_input_sheet.dart
│   │   │   ├── insight_card.dart
│   │   │   ├── disclaimer_banner.dart
│   │   │   └── error_view.dart
│   │   ├──
│   │   ├── services/
│   │   │   ├── api_client.dart    # Dio + Retrofit
│   │   │   ├── auth_service.dart
│   │   │   ├── health_service.dart
│   │   │   ├── notification_service.dart
│   │   │   ├── calendar_service.dart
│   │   │   └── offline_queue.dart
│   │   ├──
│   │   ├── providers/
│   │   │   ├── auth_provider.dart
│   │   │   ├── profile_provider.dart
│   │   │   ├── analysis_provider.dart
│   │   │   ├── chat_provider.dart
│   │   │   └── raffle_provider.dart
│   │   ├──
│   │   ├── models/
│   │   │   ├── user.dart
│   │   │   ├── supplement.dart
│   │   │   └── meal.dart

```



```

| | | └─ analysis_result.dart
| | | └─ chat_message.dart
| | |
| | └─ utils/
| |   └─ tokens.dart
| |   └─ date_utils.dart
| |   └─ formatter.dart
| |
| └─ test/
|   └─ widget_test.dart
|   └─ unit/
└─ ios/
  └─ Podfile
  └─ Runner/Info.plist    # NSHealthShareUsageDescription 등
└─ android/
  └─ build.gradle
  └─ app/
    └─ build.gradle
    └─ src/main/AndroidManifest.xml
└─ pubspec.yaml
  └─ analysis_options.yaml
|
└─ Backend/
  └─ src/
    └─ main.py            # FastAPI 진입점
    └─ config.py          # 환경변수 로딩 (CLAUDE_MODEL_ID, EMAIL_PROVIDER 등)
    |
    └─ api/
      └─ auth.py
      └─ profile.py
      └─ supplements.py
      └─ meals.py
      └─ analysis.py
      └─ chat.py
      └─ reminders.py
      └─ calendar.py
      └─ health.py
      └─ score.py
      └─ raffle.py
      └─ data_export.py
    |
    └─ algorithms/
      └─ bmi.py
      └─ activity.py      # v1~v4
      └─ weight_prediction.py  # 7-step
      └─ kdris.py         # KDRIs 록업
      └─ deficiency.py    # 결핍 판단
      └─ goal_matrix.py   # 목적별 분석
    |
    └─ ocr/
      └─ vision_adapter.py  # Cloud Vision
      └─ clova_adapter.py  # 백업 후보

```

```

├── yolo_adapter.py    # 음식 사진 인식 후보
├── llm/
│   ├── claude_client.py
│   ├── openai_client.py    # 백업
│   ├── prompts.py        # 시스템 프롬프트 + 버전 태그
│   ├── schemas.py        # Pydantic 출력 스키마
│   └── tools.py          # Tool Use 함수 정의 모음
├── agents/
│   ├── personalization_agent.py
│   ├── chat_agent.py
│   ├── evaluation_agent.py
│   ├── orchestrator.py    # 분석 알고리즘 + 3 Agent 분기 + agent_runs 로깅
│   └── memory.py          # agent_memory 갱신 로직
├── supplements/
│   ├── parser.py
│   └── matcher.py        # 영양제 CSV DB 우선 매칭 + API 보조
├── services/            # 외부 서비스 어댑터
│   ├── email.py          # SMTP / SES / NCP
│   └── storage.py        # 이미지 파일 저장
├── models/              # SQLAlchemy ORM
├── schemas/              # Pydantic 요청·응답
├── db/
│   ├── session.py
│   └── init.sql          # CREATE EXTENSION timescaledb
├── cache/                # Redis 래퍼
├── utils/
│   ├── hash.py           # SHA-256
│   ├── regex_filter.py   # 의약품 표현 사후 검수
│   └── logger.py
├── alembic.ini
├── alembic/
│   ├── env.py
│   └── versions/
├── tests/
│   ├── conftest.py
│   ├── unit/
│   └── integration/
│       └── fixtures/
├── requirements.txt
├── requirements-dev.txt
├── .env.example
├── pyproject.toml
├── Dockerfile
├── data/                # 정적 데이터
└── kdris_2020.csv

```

```

├── goal_matrix.json
├── README.md          # 출처-라이선스
├── docs/              # 추가 문서
│   ├── persona.md
│   ├── medical_review.md
│   └── compliance.md
├── github/
│   ├── CODEOWNERS      # §16 GitHub 협업 규칙 참조
│   ├── PULL_REQUEST_TEMPLATE.md
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.yml
│   │   ├── feature_request.yml
│   │   └── chore.yml
│   ├── dependabot.yml
│   └── workflows/
│       ├── ci-backend.yml
│       ├── ci-mobile.yml
│       ├── ci-docs.yml
│       └── sync-guide.yml    # §17 PG.md → guide.html 자동 동기화
├── scripts/           # 운영-자동화 스크립트
│   └── sync_guide.py      # §17 PG.md ↔ guide.html 동기화 (6중 검증)
├── docker-compose.yml  # timescale + redis
├── .gitignore
├── .pre-commit-config.yaml  # §17 로컬 commit 자동 동기화 + 위험 패턴 차단
└── .editorconfig

```

핵심 파일 책임

파일	책임
PROJECT_GUIDE.md	팀 단일 진실
guide.html	브라우저 뷰어 (HTML 틀 고정)
mobile/lib/app.dart	라우팅, 전역 상태
mobile/lib/utils/tokens.dart	디자인 토큰
mobile/ios/Runner/Info.plist	HealthKit 권한 사유
mobile/android/app/src/main/AndroidManifest.xml	Health Connect 권한
backend/src/main.py	FastAPI 진입점
backend/src/config.py	환경변수 로딩
backend/src/agents/orchestrator.py	분석 알고리즘 + 3 Agent 분기 + agent_runs 로깅
backend/src/agents/memory.py	agent_memory 갱신
backend/src/llm/prompts.py	시스템 프롬프트 + 버전 태그
backend/src/llm/schemas.py	Pydantic 출력 스키마
backend/src/llm/tools.py	Tool Use 함수 정의 모음
backend/src/services/email.py	이메일 인증 발송
backend/src/utils/regex_filter.py	의료법 표현 검수
backend/src/db/init.sql	TimescaleDB 확장 자동 설치
data/kdris_2020.csv	30종 영양소 권장 섭취량
scripts/sync_guide.py	§17 PG.md → guide.html 자동 동기화 + 6중 검증
.pre-commit-config.yaml	§17 로컬 commit 시 자동 동기화 후
.github/workflows/sync-guide.yml	§17 push 시 서버 동기화-자동 커밋, PR 시 검증

---SPLIT---

15. 팀원 작업 분담

팀원 5명. 각자 본인에게 맞는 바이브 코딩 툴(Claude Code, Codex, Cursor, Cline 등)을 자유롭게 사용한다. 단일 진실 = PROJECT_GUIDE.md .

15.1 역할 분담 (5명)

역할	담당 영역	주로 만지는 폴더
A. 프론트 리드	Flutter 라우팅·디자인 토큰·화면 통합·health 패키지	mobile/lib/app.dart, screens/, utils/tokens.dart
B. UI/UX	만성질환자 친화 UI·미리보기·챗봇 UI·응모권 화면·에러 화면	mobile/lib/widgets/, screens/chat_screen.dart, screens/raffle_screen.dart
C. AI 엔지니어	Claude API·3개 Agent·프롬프트·Tool 정의·OCR/이미지 인식 후보·의료법 검수	backend/src/llm/, agents/, ocr/, utils/regex_filter.py
D. 백엔드	FastAPI·알고리즘·DB·인증·캐싱· 보안 (JWT·RLS·AES-256) ·이메일 발송	backend/src/algorithms/, api/, models/, schemas/, db/, cache/, services/email.py
E. 데이터·도메인	KDRIs/식약처/농진청 데이터 임포트·영양제 CSV DB·병원 데이터 질문지·의료자문위 협업·컴플라이언스 검토	data/, docs/, backend/src/algorithms/kdri.py, goal_matrix.py

15.2 협업 기본 원칙 (상세 규칙은 §16)

- 일일 스탠드업: 10분 (어제·오늘·블로커)
- 같은 파일 동시 작업 X, 시작 시 채팅에 한 줄 공지
- 모든 코드 변경은 PROJECT_GUIDE.md 변경과 동기화
- 한 곳을 바꾸면 다른 곳도 같이 바뀐다 — §17.10 변경 파급 효과 표를 항상 본다

중요: 새 API 1개 추가는 단순히 `backend/src/api/foo.py` 한 파일이 아니다. §5.3 API 표, §9 호출 흐름, §14 파일 구조, §11 데이터 모델, §16.5 CODEOWNERS — 최소 5곳이 같이 바뀌어야 한다. PR 템플릿의 "코드 변경 파급 효과 점검" 체크리스트가 이를 강제한다.

15.3 바이브 코딩 툴 사용 원칙

팀원 5명이 각자 다른 툴(Claude Code, Codex, Cursor, Cline, Windsurf 등)을 써도 결과물은 같다.

원칙	설명
단일 진실 = PROJECT_GUIDE.md	모든 툴이 이 한 마크다운 파일을 우선 참조
HTML 틀은 깨지 않는다	guide.html은 <code><script id="md-source"></code> 안의 마크다운만 수정
마크다운 문법 준수	헤딩(#), 표(!), 코드블록, 인용문(>), 리스트(-, 1.)
Mermaid 다이어그램 OK	<code>```mermaid</code> 로 시작하는 코드블록은 그림으로 자동 렌더링
SPLIT 구분자 보존	세 줄짜리 SPLIT 마커(대시 3개 + SPLIT + 대시 3개)는 페이지 분할 표시. 함부로 추가/삭제 X
코드 변경 시 가이드도 갱신	알고리즘·API·스키마 바뀌면 가이드도 같이 PR
프롬프트 영역(prompts.py)은 C 담당	다른 사람은 PR 리뷰만

15.4 PROJECT_GUIDE.md 동기화 흐름

[변경 발생]

↓

[담당자] PROJECT_GUIDE.md 수정 (마크다운만)

↓

[자동] guide.html은 같은 md-source를 읽으므로 자동 반영

↓

[PR] 코드 변경 PR + 가이드 변경 PR을 같이 묶음

↓

[리뷰] 1명 이상 + 가이드 일관성 확인

↓

[머지]

15.5 각 툴별 사용 팁

툴	권장 워크플로
Claude Code	<code>claude</code> CLI에서 PROJECT_GUIDE.md 첨부 후 <code>/clear</code> 자주 사용
Codex (CLI)	<code>codex</code> 시작 시 PROJECT_GUIDE.md 컨텍스트로 먼저 로드
Cursor	<code>.cursorrules</code> 에 "always read PROJECT_GUIDE.md first" 명시
Cline / Continue	작업 시작 시 가이드 문서 첨부
Windsurf	워크스페이스 인덱싱에 PROJECT_GUIDE.md 포함

---SPLIT---

16. GitHub 협업 규칙

5명이 같은 저장소에서 충돌 없이 작업하기 위한 GitHub 운영 규칙. 모든 항목이 `.github/` 폴더 또는 저장소 설정에 반영된다.

16.1 브랜치 전략 (Trunk + Short-lived feature)

브랜치	용도	보호
<code>main</code>	배포용 (TestFlight / Play Internal에 자동 배포)	직접 push 금지, PR + 1명 리뷰 + CI 통과 필수
<code>dev</code>	통합 브랜치 (모든 feature가 먼저 머지)	직접 push 금지, PR + 1명 리뷰 + CI 통과
<code>feat/<영역>-<짧은이름></code>	개인 작업	자유. 작업 끝나면 <code>dev</code> 로 PR
<code>fix/<영역>-<짧은이름></code>	버그 수정	feat와 동일
<code>hotfix/<설명></code>	운영 중 긴급 수정	<code>main</code> 에서 분리, <code>main</code> · <code>dev</code> 양쪽 머지

영역 약어: `mobile`, `backend`, `ai`, `data`, `infra`, `docs`.

16.2 커밋 메시지 (Conventional Commits)

`<type>(<scope>): <subject>`

`<body> (선택)>`

`<footer> (선택, BREAKING CHANGE / Closes #)>`

type	의미	예
feat	새 기능	feat(ai): 챗봇 Agent add_reminder Tool 추가
fix	버그 수정	fix(backend): JWT refresh 토큰 만료 처리
refactor	동작 변경 없는 리팩토링	refactor(mobile): screens/auth 폴더로 이동
docs	문서만 변경	docs(guide): §16 GitHub 규칙 추가
test	테스트만 추가	test(algo): v4 만성질환 가중 케이스 5개
chore	빌드/설정	chore(ci): ruff 0.5로 업그레이드
style	포맷·세미콜론 등	style(backend): black 적용
perf	성능 개선	perf(cache): KDRIs 록업 메모리 캐시

규칙:

- subject 50자 이내, 한국어 OK
- body는 "왜" 중심 (코드를 보면 "무엇"은 알 수 있음)
- BREAKING CHANGE는 footer에 명시 → 메이저 버전 올림

16.3 Pull Request 규칙

PR 템플릿 (`.github/PULL_REQUEST_TEMPLATE.md`)

```
## 무엇을 했나요
<!-- 한 문장 요약 + 변경 사항 bullet -->

## 왜 했나요
<!-- 배경, 이유, 관련 이슈 (Closes #N) -->

## 어떻게 검증했나요
- [ ] 단위 테스트 추가/통과
- [ ] 로컬에서 동작 확인 (스크린샷 또는 로그)
- [ ] CI 통과

## 영향 범위
- [ ] mobile
- [ ] backend
- [ ] ai
- [ ] data
- [ ] docs / infra

## 변경 파급 효과 점검 (§17.10 표 참조)
이 변경으로 같이 갱신해야 할 곳을 모두 체크했나요?
- [ ] PROJECT_GUIDE.md 본문 갱신 (해당 섹션)
- [ ] §14 파일 구조 (새 파일/폴더 추가 시)
- [ ] §11 데이터 모델 (스키마/테이블 변경 시)
- [ ] §9 호출 흐름 다이어그램 (API/Agent 추가 시)
- [ ] §15.1 팀원 분담 / §16.5 CODEOWNERS (담당 영역 변경 시)
- [ ] §부록 A.7 pubspec.yaml 또는 requirements.txt (의존성 추가 시)
- [ ] §부록 A.2 .env 템플릿 + §16.8 시크릿 (환경변수 추가 시)
- [ ] guide.html (자동 동기화 — pre-commit이 처리, 검증만)
- [ ] 해당 없음 (코드 수정 없는 docs-only PR)

## 리뷰어가 봐야 할 곳
<!-- 핵심 파일·라인 안내 -->

## 스크린샷 / 로그 (선택)
```

PR 크기 제한

- **Small (300줄 이하 변경):** 우선. 1명 리뷰 24시간 이내
- **Medium (300~800줄):** 1명 리뷰 + 머지 전 동기 리뷰 콜 권장
- **Large (800줄 초과):** 가능하면 분할. 분할 어려우면 사전 합의 후 2명 리뷰

머지 정책

- **Squash and merge** 기본 (히스토리 깔끔)
- 단, `dev` → `main` 통합 시는 **Merge commit** (히스토리 보존)

- 머지 전 자동 검증: CI 그린 + 1명 이상 Approve + Conversation 모두 resolved

16.4 코드 리뷰 정책

항목	기준
응답 시간	평일 24시간 이내, 주말은 다음 영업일
리뷰어 지정	CODEOWNERS로 자동 지정 + 본인이 추가 지정 가능
Comment vs Request changes	의견은 Comment, 머지 막는 변경은 Request changes
Nit (사소한 의견)	<code>nit:</code> prefix → 머지 가능
큰 의견 충돌	동기 콜 또는 채팅 → 결론을 PR comment로 기록
자기 코드 리뷰	merge 전 본인이 한 번 더 diff 훑기

16.5 CODEOWNERS (`.github/CODEOWNERS`)

GitHub 사용자명은 팀 합의 후 채워 넣는다. 예시:

```

# Lemon Aid CODEOWNERS

# 기본: 모든 파일은 dev 리드(A)가 1차 리뷰
*                               @lemon-aid/dev-leads

# 모바일
/mobile/                        @member-A @member-B

# 백엔드 (알고리즘·DB·API)
/backend/src/algorithms/       @member-D
/backend/src/api/              @member-D
/backend/src/models/           @member-D
/backend/src/schemas/         @member-D
/backend/src/db/               @member-D
/backend/src/cache/            @member-D
/backend/src/services/         @member-D

# AI (Agent / 프롬프트 / OCR / 의료법 검수)
/backend/src/llm/              @member-C
/backend/src/agents/           @member-C
/backend/src/ocr/              @member-C
/backend/src/utils/regex_filter.py @member-C

# 데이터 / 도메인
/data/                         @member-E
/docs/                         @member-E
/backend/src/algorithms/kdris.py @member-E
/backend/src/algorithms/goal_matrix.py @member-E

# 가이드 문서 (모든 팀원이 함께 관리, 1명 리뷰면 OK)
/PROJECT_GUIDE.md             @lemon-aid/all
/guide.html                    @lemon-aid/all

# 인프라 (CI / Docker)
/.github/                     @member-D
/docker-compose.yml            @member-D
/Dockerfile                     @member-D

```

16.6 Issue 템플릿

Bug Report (`.github/ISSUE_TEMPLATE/bug_report.yml`)

- 제목: `[Bug] 한 줄 요약`
- 필드: 재현 단계, 기대 동작, 실제 동작, 환경(OS·기기·브라우저), 스크린샷·로그, 영향 범위 라벨

Feature Request (`.github/ISSUE_TEMPLATE/feature_request.yml`)

- 제목: `[Feat] 한 줄 요약`
- 필드: 사용자 시나리오, 기대 효과, 대안, 우선순위 (P0/P1/P2)

Chore (`.github/ISSUE_TEMPLATE/chore.yml`)

- 제목: `[Chore]` 한 줄 요약
- 필드: 작업 내용, 완료 기준

16.7 라벨 정책

카테고리	라벨
우선순위	<code>P0-blocker</code> , <code>P1-high</code> , <code>P2-normal</code> , <code>P3-low</code>
영역	<code>area:mobile</code> , <code>area:backend</code> , <code>area:ai</code> , <code>area:data</code> , <code>area:docs</code> , <code>area:infra</code>
유형	<code>type:bug</code> , <code>type:feat</code> , <code>type:refactor</code> , <code>type:test</code> , <code>type:chore</code>
상태	<code>status:in-progress</code> , <code>status:blocked</code> , <code>status:waiting-review</code> , <code>status:wontfix</code>
도움	<code>good first issue</code> , <code>help wanted</code>

16.8 시크릿 관리

비밀	보관 위치	사용처
ANTHROPIC_API_KEY	GitHub Secrets + 운영 서버 환경변수	백엔드 Claude 호출
OPENAI_API_KEY	GitHub Secrets + 운영 환경변수	백업 LLM
GOOGLE_APPLICATION_CREDENTIALS	GCP 서비스 어카운트 JSON, 1Password 또는 운영 서버 마운트	Cloud Vision OCR
MFDS_API_KEY	GitHub Secrets + 환경변수	식약처
JWT_SECRET / ENCRYPTION_KEY	운영 서버 환경변수 (Secrets에는 staging만)	인증·암호화
Firebase / Vercel 키	(해당 시) GitHub Secrets	CI 배포
TestFlight / Play Console 자격증명	Apple / Google 별도 보관	수동 배포

규칙:

- `.env` 파일은 절대 커밋 X (`.gitignore` 등록 + pre-commit으로 자동 검출)
- API 키 회전 주기: 분기 1회 또는 누출 의심 즉시
- GitHub Secrets는 `staging-*` / `prod-*` prefix로 환경 분리
- CI에서 키 사용 시 `${{ secrets.NAME }}` 형식, 절대 echo 금지

16.9 CI 워크플로 (요약, 전체 yaml은 부록 참조)

워크플로	트리거	핵심 단계
ci-backend.yml	backend/** 변경	pip install → ruff → black --check → mypy → pytest --cov
ci-mobile.yml	mobile/** 변경	flutter pub get → dart format --set-exit-if-changed → flutter analyze → flutter test
ci-docs.yml	*.md , guide.html 변경	markdown lint + SPLIT 카운트 검증 + guide.html script 태그 열림/닫힘 균형 체크
dependabot.yml	매주 월요일	의존성 PR 자동 생성 (mobile/backend/actions)

16.10 main 브랜치 보호 규칙 (저장소 설정)

GitHub Settings → Branches → Branch protection rules → main :

- Require pull request before merging (1 approval)
- Require status checks to pass before merging (ci-backend, ci-mobile, ci-docs)
- Require branches to be up to date before merging
- Require conversation resolution before merging
- Do not allow bypassing the above settings

16.11 릴리스 / 버전 태그

v0.1.0	Phase 0 종료 (W2)
v0.2.0	Phase 1 종료 (W4) — 영양제 OCR + 분석 알고리즘 동작
v0.3.0	Phase 2 종료 (W6) — 3 Agent 통합
v1.0.0	Phase 3 종료 (W8) — 시연 가능 MVP

각 태그는 GitHub Release 생성 (Changelog 자동 생성), TestFlight 새 빌드 1개 = 새 태그.

16.12 보안 사고 / 시크릿 누출 절차

1. 즉시 해당 키 회전 (Anthropic / Google / NCP 콘솔)
2. git filter-repo 또는 bfg 로 히스토리에서 제거 (push --force)
3. 팀 채팅 공지 + 영향 범위 평가
4. audit_logs / GitHub Audit log 조회 → 비정상 호출 확인
5. 24시간 내 retrospective 메모 (docs/incident-YYYY-MM-DD.md)

---SPLIT---

17. 기획서 자동 동기화

`PROJECT_GUIDE.md` (원본 마크다운)와 `guide.html` (브라우저 뷰어) 두 파일은 **항상 같은 내용**이어야 한다. 팀원이 PG.md만 고치고 push하면 GitHub에서 보이는 guide.html은 옛날 그대로다. 이를 막기 위해 자동 동기화 시스템을 두 단계로 둔다.

17.1 동작 원리

flowchart LR; A[팀원이 PG.md 수정] --> B[git commit]; B --> C[pre-commit 혹은 scripts/sync_guide.py 실행]; C --> D[guide.html 자동 갱신]; D --> E[수정분 자동 stage]; E --> F[commit 완료]; F --> G[git push]; G --> H[GitHub Actions]; H --> I[혹시 빠졌으면 | 원격에서 다시 동기화 + 자동 커밋]; I --> J[main 최신 상태 보장];

17.2 핵심 파일

파일	역할
<code>scripts/sync_guide.py</code>	PG.md 본문을 guide.html의 md-source 스크립트 블록에 주입. PG.md 안에 스크립트 닫는 태그 같은 위험 패턴이 있으면 거부
<code>.pre-commit-config.yaml</code>	로컬 commit 시 자동 동기화 + 변경분 자동 stage
<code>.github/workflows/sync-guide.yml</code>	push 시 서버에서 다시 동기화-자동 커밋 / PR 시 검증만

17.3 팀원 1회 셋업 (D1)

Python 3.11+ 설치 후
pip install pre-commit
pre-commit install

이후 `git commit` 할 때마다 자동 실행. 별도 명령 필요 없음.

17.4 일상 워크플로

- 1. PROJECT_GUIDE.md 마크다운만 수정
- 2. git add PROJECT_GUIDE.md
- 3. git commit -m "docs: §7 AI 스택에 Tool 추가 설명"
 - pre-commit 혹은 자동으로 sync_guide.py 실행
 - guide.html 변경분이 자동으로 stage-커밋에 포함됨
- 4. git push
 - GitHub Actions가 한 번 더 검증·재동기화 (안전망)

17.5 수동 실행 (필요 시)

```
# 동기화 실행
python scripts/sync_guide.py

# 동기화 안 된 상태인지 검증만
python scripts/sync_guide.py --check
```

17.6 자동 검증 항목

sync_guide.py 는 다음을 검증한다.

검증	실패 시
PG.md 안에 스크립트 닫는 태그 텍스트	거부 (HTML 깨짐 방지)
표나 인라인 코드 안에 SPLIT 마커 (대시 3개 + SPLIT + 대시 3개) 직접 표기	거부
guide.html에 NULL 바이트	자동 제거
동기화 후 SPLIT 카운트 양쪽 일치	거부
guide.html의 닫는 스크립트 태그가 정확히 3개	거부
guide.html이 닫는 html 태그로 끝남	거부

17.7 GitHub Actions 동작 시나리오

트리거	동작
<code>main / dev</code> 에 PG.md push	서버에서 <code>sync_guide.py</code> 실행 → <code>guide.html</code> 변경 시 <code>chore(docs): sync guide.html ... [skip ci]</code> 자동 커밋
PR 생성 (PG.md 또는 <code>guide.html</code> 변경)	<code>--check</code> 모드로 검증만, 동기화 안 된 PR은 CI 실패
<code>[skip ci]</code> 메시지	자동 커밋이 무한 루프 트리거하지 않도록 차단

17.8 흔한 문제 해결

증상	원인	해결
<code>pre-commit</code> 훅이 안 돈다	<code>pre-commit install</code> 안 함	한 번 실행
commit이 거부됨 (스크립트 닫는 태그 발견 메시지)	PG.md 본문에 닫는 스크립트 태그 텍스트가 들어감	다른 표현으로 우회 (예: "스크립트 닫는 태그")
commit이 거부됨 ("인라인 코드에 SPLIT 마커")	표나 본문에 ``를 직접 적음	"세 줄짜리 SPLIT 마커" 같은 풀어쓰기로 변경
GitHub Actions가 자동 커밋을 못 함	<code>permissions: contents: write</code> 누락	워크플로 yaml에 권한 추가 (현재 설정됨)
Actions가 무한 루프	<code>[skip ci]</code> 빠짐	자동 커밋 메시지에 <code>[skip ci]</code> 포함 (현재 설정됨)
두 파일이 머지 충돌	dev↔feat 머지 시 양쪽 동기화 어긋남	PG.md만 수동 머지 → <code>python scripts/sync_guide.py</code> 한 번 → commit

17.9 핵심 원칙

PROJECT_GUIDE.md를 수정한다. guide.html은 절대 직접 손대지 않는다.

guide.html을 직접 편집하면 다음 PG.md 변경 시 덮어써진다.

마크다운만 단일 진실로 유지하면 두 파일이 항상 같은 내용을 보장한다.

17.10 △ 변경 = 파급 효과 (반드시 같이 갱신)

기획서의 한 곳을 바꾸면 다른 여러 페이지가 동시에 영향을 받는다. 부분 수정만 하고 끝내면 페이지끼리 모순이 생기고, 5명이 서로 다른 가정으로 작업하게 된다. 다음 표를 보고 연쇄적으로 갱신할 위치를 항상 의식하라.

무엇을 바꿨나	같이 갱신해야 할 위치
새 API 엔드포인트 추가	§5.3 API 표 + §9 호출 흐름 다이어그램 + §14 파일구조 (<code>backend/src/api/<name>.py</code>) + §11 데이터 모델 (요청·응답 스키마) + §16.5 CODEOWNERS
새 Agent 또는 Tool 추가	§3.1 Agent 표 + §3.3 Tool 표 + §7.3 Agent별 흐름 + §9 시퀀스 다이어그램 + §14 파일구조 (<code>backend/src/agents/</code> , <code>llm/tools.py</code>)
새 화면 추가	§3.5 주요 화면 표 + §3.4 인증·온보딩 흐름 다이어그램 (해당 시) + §14 파일구조 (<code>mobile/lib/screens/</code>) + §15 팀원 분담 (담당) + §16.5 CODEOWNERS
새 DB 테이블/컬럼	§6.2 핵심 테이블 + §11 데이터 모델 스키마 + §6.3 보안 (민감정보면 AES-256 표) + Alembic 마이그레이션
새 외부 라이브러리/SDK	§4·§5·§6 기술 스택 표 + §13 사용 툴 + §부록 A.7 (pubspec.yaml) 또는 requirements.txt + §17 자동 검증 추가 시 sync_guide.py
새 환경 변수	§부록 A.2 (.env 템플릿) + §16.8 시크릿 관리 + §14 파일구조 (<code>.env.example</code>) + §21.3 (배포 환경 변수 표)
새 GitHub Actions 워크플로	§14 파일구조 (<code>.github/workflows/</code>) + §16.9 CI 워크플로 표 + §16.10 main 보호 규칙 (status checks) + §부록 A.8
새 데이터 출처	§10 데이터·외부 API + §19.7 (앱 내 출처 명시) + §14 (<code>data/</code>) + §22 참고 자료
새 의료법 표현 케이스	§19.1·20.2 표현 가이드 + <code>backend/src/utils/regex_filter.py</code> 단위 테스트 + §3.10 여러 화면
일정 변경	§1.7 일정 개요 + §12.3 주차별 매핑 + §부록 A.9 D1~D5 액션 카드
팀원 역할 변경	§15.1 역할 분담 + §16.5 CODEOWNERS + §부록 A.9 액션 카드
알고리즘 산식 변경	§8 핵심 알고리즘 + §9 호출 흐름 + 단위 테스트 (50+개) + 검증 예시값
새 페이지 신설	PG.md에 SPLIT 마커 (대시 3개 + SPLIT + 대시 3개) 추가 + guide.html 사이드바 toc-item + page-N 매핑 + 후속 페이지 번호 시프트

변경 → 영향 → 갱신 흐름

flowchart LR

```

A[한 곳 수정] → B[영향 범위 점검]
B → |17.10 표 확인| C[연쇄 위치 모두 갱신]
C → D[git commit]
D → E[pre-commit 자동 동기화]
E → F[PR 생성]
F → G[PR 템플릿 체크]
G → |"문서·구조도·다이어그램 모두 갱신?"| H[리뷰어 확인]
H → |YES| I[머지]
H → |NO| C

```

항금 원칙 3가지

- 1. "코드만 바뀌고 문서가 안 바뀐 PR"은 머지하지 않는다 — 다음 사람이 잘못된 가정으로 작업한다.
- 2. "한 페이지만 바뀌고 나머지가 그대로"는 위험 신호다 — 17.10 표를 다시 본다.
- 3. 확신이 없으면 채팅에 "\$N 바꾸려는데 어디 같이 봐야 할까?" 물어본다 — 5분 질문이 5시간 디버깅을 막는다.

---SPLIT---

18. 리스크 & 대응

18.1 8대 리스크 (R1~R8)

ID	리스크	영향도	대응
R1	만성질환자 디지털 친화도 ↓	중	큰 글씨·3탭 이내·쉬운 말 카피
R2	의료법·약사법 위반	매우 높음	면책 표준 문구 3종 / 금지 표현 사전+사후 검수 / 의료자문위 검토
R3	경쟁자 진입 (필라이즈 후속)	중	의료기관 연계(LDB) + 만성질환 v4 가중
R4	만성질환 데이터 부족	중	멘토 질문지로 데이터 범위 확정, 향후 LDB 인터페이스 설계서
R5	OCR 정확도 미달 (<85%)	중	OCR/이미지 인식 후보 비교 / 사용자 수정 입력
R6	산출식 임상 한계	중	Phase 3에 Hall 동적 모델 검토
R7	개인정보 유출	매우 높음	AES-256 / RLS / TLS 1.3 / 감사 로그 / 즉시 삭제 (D 담당)
R8	DTx 오인	낮음	"치료" 표현 X, "관리·참고"로 통일

18.2 운영 리스크

리스크	대응
LLM 응답 시간	Streaming + 로딩 스켈레톤 + 캐시 3단계
LLM 응답 부정확	항상 미리보기 후 사용자 수정
LLM 비용 폭주	캐싱 + 분당 5회/일당 50회 + max_tokens
API 키 노출	모바일 빌드물에 키 X, 백엔드 환경변수만
OCR/이미지 인식 후보 비용 초과	캐싱 50%+ 절감 + 후보 기술 비교
Health Connect 검토 지연	iOS HealthKit 우선 진행, Android는 수동/mock 데이터 검토
TestFlight 외부 심사 24~48시간	W7 첫 빌드 사전 업로드, 내부 테스터 풀백
8주 일정 타이트	분석 알고리즘·평가에 집중, 챗봇·개인화는 단순화 가능

18.3 비상 대응 시나리오

상황	대응
자해/자살 의도 감지	109/1577-0199 안내, 다이어트 권고 즉시 비표시
식이장애 의심	한국섭식장애협회 안내
심각 건강 이상	1339 응급의료, 일반 권고 일시 중단
데이터 유출	1시간 차단 → 24시간 신고 → 72시간 분석 → 1개월 보고

원칙: 3개 Agent 중 일부가 단순화되어도 코어(영양제 분석 알고리즘 + 5종 출력)는 살아남는다. 발표는 무조건 성립한다.

---SPLIT---

19. 컴플라이언스 & 안전선

본 서비스는 진단·치료 서비스가 아니라 웰니스 기반 건강관리 보조 서비스로 정의한다. 의료법·약사법·건강기능식품법·개인정보보호법 5개 법령을 준수.

19.1 표현 가이드 — 절대 금지 vs 허용

절대 금지 (의료법·약사법)

- 1. "OO질환입니다" — 진단
- 2. "OO약을 드세요" — 처방
- 3. 특정 의료기관 알선
- 4. 의사·약사 오인 표현
- 5. 치료 효과 단정 보장
- 6. 특정 의약품 추천·이름 명시
- 7. 의약품의 건기식화 광고

허용

- 1. KDRIs 권장량 안내
- 2. 일반 식이·운동 권고
- 3. 영양소 결핍 가능성 (질병명 X)
- 4. "전문가와 상담을 권장합니다"
- 5. 일반인 대상 영양·운동·체중 정보
- 6. 식약처 기능성 인정 원료 안내
- 7. 영양제 라벨 단순 표시
- 8. "약사와 상의" 안내
- 9. "주의 가능성", "권장량 대비", "전문가 상담 권장" 형식

19.2 표현 가이드 — 위반 → 대체

위반	대체
"당신은 빈혈입니다"	"철분 섭취량이 권장량보다 낮습니다"
"비타민 D가 결핍되었습니다"	"비타민 D 섭취가 권장량의 35% 수준입니다"
"당뇨 위험군입니다"	"혈당 관련 영양소 관리에 주의가 필요할 수 있습니다"
"OO 비타민C를 드세요"	"비타민 C 섭취량을 늘리는 것이 권장됩니다"
"이대로면 비만이 됩니다"	"현재 추세 시 1개월 후 체중이 약 1.2 kg 증가할 수 있습니다"
"운동 부족으로 병이 옵니다"	"권장 걸음수의 60% 수준으로 활동하고 계십니다"

19.3 면책 고지 표준 문구 3종

다음 문구는 초안이며, 세부 문안은 추후 확정한다.

메인 (모든 권고 화면 하단)

"본 서비스에서 제공하는 정보는 일반적인 건강 관리를 위한 참고 자료이며, 의사·약사·영양사의 전문적 진단이나 처방을 대체하지 않습니다. 증상이 있거나 만성질환을 앓고 계신 경우, 반드시 전문가와 상담하시기 바랍니다."

영양제

"영양제는 의약품이 아니며, 질병의 예방이나 치료를 보장하지 않습니다. 약을 복용 중이신 경우, 영양제와의 상호작용에 대해 의료진과 상담하세요."

체중 예측

"체중 변화 예측은 평균적인 수치를 바탕으로 산출되며, 개인의 대사·체질·생활습관에 따라 결과가 달라질 수 있습니다. 급격한 체중 변화는 건강에 해로울 수 있으니 의료진과 상담하세요."

19.4 자동 검수 장치 (4단계)

flowchart LR
A[1. LLM 시스템 프롬프트
금지어 명시] → B[2. Pydantic 스키마 검증]
B → C[3. 정규식 사후 필터
최대 3회 재생성]
C → D[4. 사용자 미리보기
승인 후 저장]

Agent와 LLM 응답에는 주의·권장 표현과 필요한 출처를 함께 표시한다. 사진 분석 화면에는 출처를 직접 노출하지 않고, 챗봇 대화 안에서 출처를 표시한다.

19.5 개인정보보호법 — 민감정보 처리

분류	항목	동의
민감정보 또는 엄격 처리 대상 (별도 동의 필수)	만성질환·복약·검진기록·혈당수·심박수·건강 관련 이미지	항목별 체크
일반정보	이름·이메일·나이·성별·키·몸무게	통합 동의

처리 원칙:

- 별도 동의 UI (필수/선택 구분 + 사용 목적 표시)
- 건강 관련 이미지는 영양제·음식 사진도 엄격한 저장·삭제 기준으로 처리
- 가명정보 처리
- 데이터 주체 5권리 (열람 / 정정·삭제 / 처리정지 / 동의 철회 / 전체 삭제 요청 + 3개월 복구 가능 기간 후 완전 삭제)
- AES-256 + TLS 1.3 + 감사 로그

19.6 DTx 해당 여부

결론: 비의료 건강관리서비스, 식약처 허가 불필요.

3가지 DTx 요건 모두 X:

- 임상 근거 X (KDRIs 기반 일반 권고)
- 질병 치료 목적 X (관리·참고)
- 특정 질환자 프로토콜 X (일반인 대상)

가드: "치료" 표현 X, "관리·참고" 통일.

19.7 데이터 출처 명시

사진 분석 결과 화면에는 출처를 직접 표시하지 않는다. 챗봇/LLM 대화 안에서 필요한 경우 다음 출처를 표시한다.

- 한국영양학회 KDRIs 2020 (보건복지부)
- 영양제 CSV DB
- 식약처 식품영양성분 Open API
- 식약처 건강기능식품 원료 DB
- 농촌진흥청 국가표준식품성분표
- AI Hub 음식 이미지 (NIA, 비상업 학술)

19.8 Phase별 컴플라이언스 체크리스트

Phase	체크
Phase 0~1	표준 디스클레이머 / LLM 시스템 프롬프트 + 금지어 검출
Phase 2	별도 동의 UI / 디스클레이머 / AES-256 / TLS 1.3 / PHI 감사 로그
Phase 3	의료자문위 / 식약처 표현 검수 / DTx 사전 검토 / 위급 신호 감지
Phase 4	법무 검수 진단·처방 0건 / 챗봇 출처 표시 / App Privacy 라벨 / Data Safety / 개인정보처리방침

---SPLIT---

20. 시연 시나리오

20.1 시나리오 1 — 김건강 (만성질환자, 핵심 어필)

[상황]

52세 남성, 고혈압 진단 2년차, 당뇨 전단계.
혈압약 1종 + 영양제 4종 복용. BMI 26.5.

[시연 흐름]

1. 앱 진입 → 김건강 프로필 자동 로드 (만성질환·복약 표시)
2. 카메라 → 영양제 4종 라벨 차례로 촬영
→ 분석 알고리즘이 약 30~60초 안에 4종 모두 분석
※ 발표 전 캐시 워밍 완료 시 30초 이내
3. 5종 출력 대시보드 진입
- 부족: "비타민D 35%" / 과다: "비타민B6 1.4배 (UL 근접)"
- 권장: KDRIs 50대 남성 + 고혈압 보정값
- 체중 예측: 30일 후 82.8 kg (-1.2 kg)
- 활동 권고: "당뇨+고혈압 가중으로 7,000보가 9,000보 가치"
- 목적별: "간기능에 밀크씨슬 적정, 추가 보충 불필요"
4. 평가 Agent 점수: "오늘 점심 78점 — 단백질 충분, 나트륨 약간 높음"
5. 챗봇 진입 → "이 영양제 계속 먹어도 돼?"
→ "비타민B6가 권장량의 1.4배입니다. 종합비타민에도 B6가 들어 있어 중복일 수 있습니다. 약사와 상담을 권장드립니다."
6. 챗봇에 "내일부터 매일 아침 8시에 혈압약 알림" 입력
→ Tool Use로 알림 등록 → 미리보기 → 사용자 승인 → 저장
7. 응모권 화면: "오늘 사진 4장 등록 완료, 응모권 1개 추가"

20.2 시나리오 2 — 박직장 (예방 단계)

[상황]

38세 남성, 콜레스테롤·공복혈당 경계.
영양제 2종, 평일 5,000보 미만.

[시연 흐름]

1. 점심 사진 촬영 → 분석 알고리즘: "김치찌개·공깃밥·계란말이"
2. 5종 출력: "탄수화물 충분, 단백질 부족, 식이섬유 부족"
3. 체중 예측: "지금 추세 시 3개월 후 85 kg" → 경고
4. 챗봇: "다이어트하려면 어떻게 해야 돼?"
→ "현재 활동량이 권장의 60% 수준입니다. 하루 8,000보 목표, 저녁 식사에 채소 1접시 추가를 권장드립니다."
5. 사용자가 "내일부터 저녁 7시 산책 알림" 등록
6. 응모권 + 점수 화면

20.3 시나리오 3 — 챗봇 알림 등록 (Tool Use 어필)

1. "매일 아침 8시 혈압약, 저녁 8시 종합비타민 알림 맞춰줘"

→ 챗봇 Agent가 의도 파악, 2개 알림 동시 등록

→ 미리보기 화면 → 승인
2. "다음 진료가 6월 3일 화요일 오후 2시야. 캘린더에 넣어줘"

→ 캘린더 등록 미리보기 → 승인 → 시스템 캘린더 반영
3. "오늘 비타민D 먹었어"

→ 영양제 섭취 기록 추가 + 응모권 카운트

20.4 시연 안전장치

리스크	대응
발표장 와이파이 불안정	사전 데모 데이터 시드 + 김건강 영양제 4종 사진 캐시 워밍
LLM 응답 지연	캐시 히트 시연 우선, 미스는 백업 영상
카메라 인식 실패	미리 준비한 라벨 사진 + 수동 입력 폴백
백엔드 장애	발표 PC에서 로컬 docker-compose 직접 시연
TestFlight 외부 그룹 심사 미통과	내부 테스터(최대 100명) 그룹 폴백
청중 참여	QR 코드 → TestFlight 링크 + 베타 가입

---SPLIT---

21. 앱 배포 & 시연

21.1 배포 전략

- flowchart LR
- M[Flutter iOS+Android] → TF[TestFlight / Play Internal]
- B[FastAPI + Docker] → CL[NCP/AWS/GCP 학생 크레딧]
- TF → D[실기기 + QR]
- CL → D

21.2 모바일 배포

플랫폼	채널	비용	비고
iOS	TestFlight (베타)	\$99/년	외부 테스터 1만 명, 심사 24~48h
Android	Google Play Internal Testing	\$25 (1회)	100명 내부 테스터
시연용	발표자 폰 직접 빌드	0원	adb install / Xcode

21.3 백엔드 배포

- [1] Docker Compose 로컬 검증
- [2] 클라우드 (NCP / AWS / GCP) 학생 크레딧
- [3] HTTPS + 도메인 (api.lemonaid.dev 등)
- [4] 환경 변수 (백엔드 환경에만):
 - ANTHROPIC_API_KEY
 - CLAUDE_MODEL_ID
 - GOOGLE_APPLICATION_CREDENTIALS
 - MFDS_API_KEY
 - DATABASE_URL
 - REDIS_URL
 - JWT_SECRET (openssl rand -hex 32)
 - ENCRYPTION_KEY (openssl rand -base64 32)
 - EMAIL_PROVIDER (smtp / ses / ncp)
 - SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS (개발 시)
- [5] 모바일 앱은 API 도메인만 알면 됨, 키는 백엔드에만

21.4 발표장 시연 시나리오

1. PC 빔프로젝터: 슬라이드 (한 줄 요약 + 차별화 5개)
2. 본인 iPhone 꺼냄 → "TestFlight 베타입니다"
3. 김건강 페르소나 시연 (3분):
 - 영양제 4종 카메라 (캐시 워밍 → 30초 이내)
 - 5종 출력 대시보드
 - 챗봇 알림 등록
 - 응모권 누적
4. 청중 참여 — QR 코드 → TestFlight 링크
5. 마무리: 차별화 5개 + LDB 연계 가능성

21.5 시연 안전장치

리스크	대응
발표장 와이파이 불안정	사전 시드 + 캐시 워밍 + 모바일 데이터 백업
백엔드 장애 (희박)	발표 30분 전 헬스 체크, 로컬 백엔드 + 시뮬레이터 폴백
AI 응답 지연	캐시 히트 시연 우선, 미스는 백업 영상
iOS 폴스크린 안 됨	TestFlight 정식 빌드
도메인 타이핑	QR 코드
카메라 인식 실패	미리 촬영한 라벨 사진 갤러리 선택
TestFlight 심사 미통과	내부 테스터(100명) 폴백

21.6 v2 — 정식 출시 단계

단계	작업
MVP+	베타 30~100명, 정량 SUS, OCR 정확도 검증, 의료자문위 정식 검토
정식 출시	App Store / Google Play 정식 등록, ISMS-P 인증 추진
LDB 통합	5~10개 의료기관 시범, FHIR KR Core 표준
글로벌	일본·동남아 i18n

---SPLIT---

22. 참고 자료

22.1 데이터 · 가이드라인

- KDRI's 2020 (한국영양학회) — <https://www.kns.or.kr>
- 식약처 식품영양성분 Open API — <https://various.foodsafetykorea.go.kr>
- 식약처 건강기능식품 원료 DB — <https://www.foodsafetykorea.go.kr>
- 농촌진흥청 국가표준식품성분표 — <https://koreanfood.rda.go.kr>
- AI Hub 한국 음식 이미지 — <https://aihub.or.kr>
- 보건복지부 비의료 건강관리서비스 가이드라인 (2차)
- KISA ISMS-P 인증 기준 — <https://isms.kisa.or.kr>

22.2 AI · LLM

- Anthropic Claude API — <https://docs.claude.com>
- OpenAI API — <https://platform.openai.com/docs>
- Google Cloud Vision API — <https://cloud.google.com/vision/docs>
- Naver CLOVA OCR — <https://www.ncloud.com>

22.3 기술 스택

- Flutter — <https://flutter.dev/docs>
- Riverpod — <https://riverpod.dev>
- Dio — <https://pub.dev/packages/dio>
- health 패키지 — <https://pub.dev/packages/health>
- fl_chart — https://pub.dev/packages/fl_chart
- FastAPI — <https://fastapi.tiangolo.com>
- Pydantic — <https://docs.pydantic.dev>
- SQLAlchemy — <https://docs.sqlalchemy.org>
- PostgreSQL — <https://www.postgresql.org/docs>
- TimescaleDB — <https://docs.timescale.com>
- Redis — <https://redis.io/docs>
- Alembic — <https://alembic.sqlalchemy.org>

22.4 법규 · 컴플라이언스

- 의료법·약사법·건강기능식품법·개인정보보호법 — <https://www.law.go.kr>
- 개인정보보호위원회 가명정보 처리 가이드라인 — <https://www.pipc.go.kr>

22.5 발주처

- (주)레몬헬스케어 — <https://www.lemonhealthcare.com>

---SPLIT---

23. 최종 메시지

우리 프로젝트는 **음식과 영양제 분석을 출발점**으로 한다.

하지만 최종적으로는 만성질환자의 **병원성 데이터·복약 정보·식단·건강 데이터**를 연결해 개인화된 **식단관리 판단과 영양제 관리 피드백**을 제공하는 **AI Agent 서비스**를 제안한다.

이는 레몬헬스케어가 목표로 하는 **생애 전 주기 개인화 헬스케어**로 가기 위한 작은 MVP이자, **건강의신에 적용 가능한 서비스 참조 모델**이다.

23.1 한 문장 요약

사진 한 장으로 시작해, 병원 기록을 기억하는 AI Agent로 끝난다.
음식·영양제 분석은 입구일 뿐, 진짜 가치는 만성질환자의 일상과 의료 데이터를 잇는 자리에 있다.

23.2 발주처에게 — 이 프로젝트가 남기는 것

항목	결과물
검증된 알고리즘	v1~v4 활동점수 / 7-step 체중 예측 / KDRIs 결핍 판단 / 영양제 OCR-LLM 파이프라인
재사용 가능한 코드	FastAPI 모듈 / Pydantic 스키마 / 3개 Agent 시스템 프롬프트 / Flutter 화면 위젯
컴플라이언스 가드	의료법 표현 검수 함수 / 면책 표준 문구 3종 / 민감정보 동의 UI
LDB 통합 인터페이스 설계	향후 건강의신과 LDB 의료기관 데이터를 연결할 출발점
정성 사용성 결과	내부 테스터 5명 + 멘토·자문위 3명 의견서

23.3 팀에게 — 우리가 만든 것

5명이 각자 다른 바이브 코딩 툴 쓰면서도 **하나의 가이드 문서(PROJECT_GUIDE.md)**로 같은 그림을 보고 작업할 수 있게 했다. 코드는 Flutter + FastAPI + Claude Agent의 협업이지만, 그 시작과 끝은 마크다운 문서 한 장에서 출발한다.

다음 팀이 와도 이 가이드만 읽으면 우리가 어디에서 출발했고, 어디로 가려 했는지 알 수 있다.
그게 이 문서의 목적이다.

---SPLIT---

부록 A. D1 즉시 시작 가이드 — 5명이 지금 바로 작업 시작

A.0 가장 먼저 (모든 팀원 공통, 30분)

- 이 문서를 첫 페이지부터 끝까지 한 번 읽기 (스크롤만 1회). 30분 정도 걸린다.
- 본인의 바이브 코딩 툴(Claude Code / Codex / Cursor / Cline / Windsurf 중 하나)을 열고, 첫 명령으로 **PROJECT_GUIDE.md**를 컨텍스트에 로드한다.
- GitHub 저장소를 fork 또는 clone하고, 본인 이름으로 **feat/<영역>-setup** 브랜치를 만든다.
- 팀 채팅에 "[본인 역할] 시작합니다" 메시지 + 오늘 할 일 한 줄.

A.1 저장소 클론 + 환경 셋업

```
# 1) 저장소 클론
git clone https://github.com/Lemon-Aid-KDT/Lemon-sin.git
cd Lemon-sin

# 2) 백엔드 의존성
cd backend
python -m venv .venv

# 가상환경 활성화
# macOS/Linux: source .venv/bin/activate
# Windows cmd: .venv\Scripts\activate.bat
# Windows PS: .venv\Scripts\Activate.ps1

pip install -r requirements.txt -r requirements-dev.txt
cp .env.example .env

# 3) Docker Compose (timescale + redis + mailhog)
cd ..
docker compose up -d

# 4) DB 마이그레이션 (D1 첫 사람만 1회 init)
cd backend
alembic init -t async alembic
alembic revision --autogenerate -m "init"
alembic upgrade head

# 5) 백엔드 개발 서버
uvicorn src.main:app --reload --port 8000

# 6) 모바일 (별도 터미널)
cd ../mobile
flutter pub get
flutter run
```

A.2 환경 변수 (backend/.env)

```
DATABASE_URL=postgresql+asyncpg://lemon:lemon@localhost:5432/lemon
REDIS_URL=redis://localhost:6379/0

ANTHROPIC_API_KEY=sk-ant-...
CLAUDE_MODEL_ID=claude-sonnet-latest
OPENAI_API_KEY=sk-...
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
CLOVA_OCR_API_KEY=...

MFDS_API_KEY=...

JWT_SECRET=...           # openssl rand -hex 32
ENCRYPTION_KEY=...       # openssl rand -base64 32

EMAIL_PROVIDER=smtp      # smtp | ses | ncp
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_USER=
SMTP_PASS=

ENVIRONMENT=development
LOG_LEVEL=DEBUG
```

A.3 docker-compose.yml 핵심부

```
version: '3.9'

services:
  db:
    image: timescale/timescaledb:latest-pg16
    environment:
      POSTGRES_USER: lemon
      POSTGRES_PASSWORD: lemon
      POSTGRES_DB: lemon
    volumes:
      - ./backend/src/db/init.sql:/docker-entrypoint-initdb.d/init.sql
      - lemon-pg-data:/var/lib/postgresql/data
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    volumes:
      - lemon-redis-data:/data

  mailhog:
    # 개발용 SMTP UI
    image: mailhog/mailhog
    ports: ["1025:1025", "8025:8025"]

volumes:
  lemon-pg-data:
  lemon-redis-data:
```

`backend/src/db/init.sql` :

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
```

A.4 첫 커밋 전 체크리스트

- .env 작성됐는가
- docker compose ps에 timescale + redis + mailhog 동작
- uvicorn이 8000 포트에서 뜨는가
- http://localhost:8000/docs 에 Swagger UI 나오는가
- flutter run으로 시뮬레이터 동작
- pre-commit install 완료

A.5 W1~W2 종료 시점에 있어야 할 것

- FastAPI 빈 셀 + 인증 라우터

- PostgreSQL/TimescaleDB/Redis Docker 환경
- Alembic 초기 마이그레이션
- Flutter 빈 셀 + 라우팅 + 디자인 토큰
- 카메라 화면 + 갤러리 선택
- backend/src/llm/claude_client.py 빈 함수 시그니처 (§A.6)
- backend/src/llm/tools.py 5개 Tool 정의
- backend/src/agents/orchestrator.py + memory.py 빈 셀
- backend/src/algorithms/bmi.py + 단위 테스트
- backend/src/services/email.py SMTP 발송 PoC
- Android Health Connect 연동 검토

A.6 D1 코드 시그니처 (5명 공통 합의, 변경 시 PR 필수)

```
# backend/src/llm/claude_client.py
async def call_claude(
    *,
    system: str,
    messages: list[dict],
    tools: list[dict] | None = None,
    max_tokens: int = 1024,
    timeout_s: int = 12,
) → ClaudeResponse: ...

# backend/src/agents/orchestrator.py
async def run_full_analysis(
    user_id: int,
    supplements: list[UploadFile],
    meal: MealInput | None,
) → FullAnalysisResult: ...

# backend/src/agents/memory.py
async def update_memory(user_id: int, evaluation: EvaluationResult) → None: ...

# backend/src/llm/tools.py
TOOLS = {
    "extract_supplement_facts": {...},
    "add_reminder": {...},
    "add_calendar_event": {...},
    "log_supplement_intake": {...},
    "explain_deficiency": {...},
}

# backend/src/algorithms/bmi.py
def classify_bmi(height_cm: float, weight_kg: float) → BmiClass: ...

# backend/src/algorithms/activity.py
def compute_v4(profile: Profile, daily: DailyActivity) → ActivityScore: ...

# backend/src/utls/regex_filter.py
def check_forbidden_terms(text: str) → ForbiddenCheck: ...

# backend/src/services/email.py
async def send_verification_email(to: str, token: str) → None: ...
```

A.7 pubspec.yaml 의존성 핵심부

```
name: lemon_aid
description: AI 헬스케어 / 건강관리 플랫폼
publish_to: 'none'
version: 1.0.0+1
```

```
environment:
  sdk: ">=3.4.0 <4.0.0"
  flutter: ">=3.24.0"
```

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.5.1
  go_router: ^14.2.0
  dio: ^5.4.3
  retrofit: ^4.1.0
  image_picker: ^1.1.2
  camera: ^0.11.0
  health: ^11.0.0
  isar: ^3.1.0
  isar_flutter_libs: ^3.1.0
  fl_chart: ^0.68.0
  flutter_local_notifications: ^17.2.0
  add_2_calendar: ^3.0.1
  freezed_annotation: ^2.4.4
  json_annotation: ^4.9.0
  intl: ^0.19.0
```

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.11
  freezed: ^2.5.7
  json_serializable: ^6.8.0
  retrofit_generator: ^8.2.0
  flutter_lints: ^4.0.0
```

A.8 GitHub Actions CI 핵심 step

워크플로	실행 단계
ci-backend.yml	<code>pip install -r requirements.txt -r requirements-dev.txt</code> → <code>ruff check .</code> → <code>black --check .</code> → <code>mypy src</code> → <code>pytest --cov</code>
ci-mobile.yml	<code>flutter pub get</code> → <code>dart format --set-exit-if-changed .</code> → <code>flutter analyze</code> → <code>flutter test</code>
ci-docs.yml	<code>markdownlint PROJECT_GUIDE.md README.md</code> → SPLIT 카운트 검증 → guide.html script 태그 균형 체크 + <code>python scripts/sync_guide.py --check</code>

A.9 5명별 D1 ~ D5 액션 카드

A — 프론트 리드 (Flutter 환경 + 라우팅)

D1: Flutter 프로젝트 init, pubspec.yaml 의존성 (§A.7), main.dart + app.dart 셸
D2: go_router 7화면 라우팅, Splash → Login → Onboarding → Home
D3: utils/tokens.dart 디자인 토큰 (라이트 단독), Pretendard 폰트
D4: health 패키지 권한 요청 화면 + Info.plist / AndroidManifest.xml 작성
D5: 카메라 화면 (image_picker + camera) MVP

B — UI/UX (위젯 + 미리보기)

D1: Figma 와이어프레임 (Splash, Login, Signup, Consent)
D2: 화면 토큰 적용, widgets/disclaimer_banner.dart, error_view.dart
D3: supplement_preview.dart 미리보기 위젯 (분석 결과 + 사용자 수정)
D4: ai_input_sheet.dart 챗봇 입력 시트
D5: insight_card.dart, raffle_screen.dart UI

C — AI 엔지니어 (Claude + Tool + 검수)

D1: backend/src/llm/claude_client.py 빈 시그니처 (§A.6) + 기본 호출 테스트
D2: backend/src/llm/prompts.py 3개 Agent 시스템 프롬프트 v0
D3: backend/src/llm/tools.py 5개 Tool 정의 (§3.3)
D4: backend/src/llm/schemas.py Pydantic 출력 스키마
D5: backend/src/llm/regex_filter.py 의뢰법 검수 + 단위 테스트

D — 백엔드 (FastAPI + DB + 인증 + 보안)

D1: docker-compose.yml + backend/src/db/init.sql + alembic init
 D2: backend/src/main.py + config.py + db/session.py
 D3: backend/src/api/auth.py (signup, login, refresh, JWT)
 D4: backend/src/algorithms/bmi.py + activity.py + 단위 테스트
 D5: backend/src/services/email.py SMTP PoC + RLS 정책 SQL

E — 데이터 · 도메인 (KDRIs + 식약처 + 컴플라이언스)

D1: data/kdris_2020.csv 임포트 스크립트 작성
 D2: backend/src/algorithms/kdris.py 록업 함수 + 단위 테스트
 D3: 식약처 식품영양성분 Open API PoC (FastAPI 어댑터)
 D4: data/goal_matrix.json 작성 (눈/간/피로 §8.7 표 그대로)
 D5: docs/medical_review.md 의료자문위 질문 초안 + Android Health Connect 검토 진행

A.10 팀 공유 채팅 메시지 템플릿

☒ Lemon Aid W1 작업 시작합니다

[저장소] <https://github.com/Lemon-Aid-KDT/Lemon-sin>
 [가이드] PROJECT_GUIDE.md (단일 진실) / guide.html (브라우저로 열기)
 [브랜치] feat/<영역>-<짧은이름> · main 직접 푸시 X · PR 1명 리뷰

작업 분담 (§A.9 D1 액션 카드 참조):

- A (프론트 리드): Flutter init + 라우팅
- B (UI/UX): Figma 와이어프레임
- C (AI 엔지니어): claude_client.py 빈 시그니처
- D (백엔드): docker-compose + alembic init
- E (데이터·도메인): KDRIs CSV 임포트 + 영양제 CSV DB + Health Connect 검토

질문/블로커는 채팅 즉시. 매일 18시 스탠드업 10분.

- △ 바이브 코딩 툴 사용 시 PROJECT_GUIDE.md 먼저 읽어주세요.
- △ guide.html은 직접 수정하지 마세요. PROJECT_GUIDE.md만 수정 → pre-commit이 자동 동기화.
- △ 코드 시그니처(A.6)는 합의된 인터페이스. 변경 시 PR 필수.
- △ GitHub 협업 규칙은 §16 참조. 자동 동기화는 §17 참조.

A.11 막힐 때 / 도움 받을 때

상황	어디 보기
환경 셋업이 안 됨	§A.1 명령 순서 / Docker 로그 / 채팅에 에러 그대로
API 키 받는 법 모름	Anthropic Console / Google Cloud Console / 채팅에 멘토 호출
다른 팀원 코드와 충돌	§16 GitHub 규칙 + 채팅에서 동기 콜
알고리즘 산식이 헷갈림	§8 핵심 알고리즘 / 가이드 PPT 예시값과 비교
의료법 표현이 걱정	§19.2 위반→대체 표 / E에게 검토 요청
분석 알고리즘 + 3개 Agent 흐름이 헷갈림	§3.1 / §7.3 / §9 호출 흐름
LLM 비용이 무서움	§7.6 가드레일 / 캐시 적중률 점검
발표 직전인데 백엔드 죽음	§20.4 / §21.5 시연 안전장치
guide.html이 PG.md와 다르게 보임	§17 기획서 자동 동기화

본 문서는 Lemon Aid 팀 공유용 가이드라인입니다.
변경 시 PROJECT_GUIDE.md만 수정하면 guide.html에 자동 반영됩니다 (§17 자동 동기화).